

Sicurezza informatica: → abilità di un sistema di **proteggere informazioni**, risorse ed il sistema stesso, rispetto alle nozioni di:

- **Confidentialità (confidentiality)**
Assicurare che le informazioni non siano accessibili ad utenti non autorizzati
- **Integrità (integrity)**
Assicurare che le informazioni non siano alterabili da persona non autorizzate (in maniera invisibile agli utenti autorizzati)
- **Autenticazione (authentication)**
Assicurare che gli utenti siano effettivamente chi dichiarano di essere
- **Disponibilità (availability)**
Assicurare che un sistema sia operativo e funzionale in ogni momento (non deny-of-service)
- **Controllo degli accessi (access control)**
Assicurare che gli utenti abbiano accesso a tutte le risorse ed a tutti i servizi cui sono autorizzati e solo a questi
- **Non ripudio (non-repudiation)**
Assicurare che il mittente di un messaggio non possa negare il fatto di aver spedito il messaggio
- **Privatezza (privacy)**
Assicurare che gli utenti possano controllare quali informazioni su di lui vengono raccolte, come vengono usate, chi le usa, chi le mantiene, e per quale scopo vengono usate

Alla base della **comunicazione di rete** c'è il **protocollo TCP/IP**, adottato da molti anni. Questo protocollo è diventato uno standard non perché sia stato pianificato a tavolino, ma perché è risultato essere il più utilizzato. Inizialmente, si stavano studiando altri standard, come il **modello ISO**, ma non sono mai stati adottati poiché, quando venivano sviluppati, il TCP/IP era già diffuso.

Il protocollo IP, che è stato aggiornato, ha portato alla creazione dell'**IP versione 6**, che include miglioramenti per la **sicurezza**, cosa che l'IP versione 4 non aveva. Gli **attori principali** in una rete sono: i **PC** connessi tramite **Wi-Fi** o cavo, i **server**, i **router** e altre macchine connesse.

La **comunicazione di rete** è un problema complesso e, come in molti altri campi, si cerca di dividerlo in parti per semplificarlo. Per esempio, quando si deve inviare un messaggio da una macchina a un'altra, si suddivide il problema in sotto-problemi. Questo è il motivo per cui lo studio delle reti avviene a **livelli**. Ogni livello affronta un aspetto specifico della comunicazione. Un esempio è la comunicazione tramite **HTTP**.

HTTP (HyperText Transfer Protocol) è un protocollo che regola la comunicazione tra il **browser** e il **server** per il caricamento delle pagine web. **Chi conosce il significato di HTTP?** HTTP è il metodo utilizzato dalle macchine per comunicare tra loro. Per spiegare cosa sia un **protocollo**, possiamo fare un esempio con il **telefono**: quando si alza la cornetta, si dice "pronto", è una **regola di comunicazione**, proprio come in HTTP, dove c'è un insieme di regole per permettere la comunicazione tra due dispositivi.

Le comunicazioni in rete avvengono attraverso **protocolli**, proprio come la comunicazione telefonica. Quando parliamo al telefono, seguiamo un insieme di regole, un **protocollo**, per poter comunicare. Lo stesso accade per le comunicazioni in rete, dove il **protocollo TCP/IP** è quello fondamentale.

In una **rete** possiamo avere architetture **client-server** o **P2P**. La principale differenza tra client e server è che il **server** è sempre acceso e risponde alle richieste del **client**, che può essere acceso solo quando necessario. Per esempio, il sito **www.ipm.com** è sempre attivo, mentre il vostro **browser** si attiva quando lo aprete.

Il **protocollo TCP** garantisce che i messaggi vengano inviati nel giusto ordine, senza errori e integri, tra il **client** e il **server**. Inoltre, si occupa di gestire la **congestione** della rete, proprio come accade nel traffico

stradale, dove si rallenta il flusso per evitare ingorghi. Quando ci sono troppi pacchetti in circolazione, il **TCP** riduce la velocità di invio per evitare il sovraccarico.

Un altro aspetto del **protocollo TCP** è il **controllo di flusso**, che regola la velocità con cui i pacchetti vengono inviati, in base alla capacità del **server** di gestirli. Se il server è troppo lento, non ha senso inviare troppe richieste contemporaneamente.

Esistono anche altri protocolli a livello di trasporto, come il **UDP**, che è più semplice e non ha tutte le caratteristiche del **TCP**, ma viene comunque utilizzato per specifiche applicazioni.

A livello di **trasporto**, il **protocollo IP** si occupa di **instradare** i pacchetti attraverso la rete. Ogni pacchetto sa qual è il percorso da seguire per arrivare alla sua destinazione. Questo processo di **routing** è fondamentale per la comunicazione tra macchine in una rete.

Il livello **Ethernet** è quello che gestisce la comunicazione a livello locale, come quella tra il vostro **PC** e il **router**, o tra dispositivi all'interno di una **rete locale**.

Le **comunicazioni wireless** possono presentare problemi di **sicurezza**, che verranno approfonditi in seguito.

Problemi di Sicurezza in Rete e Diagnostica del Traffico

Quando parliamo di **Wi-Fi**, ci riferiamo all'access point, un dispositivo che consente di connettersi a **Internet** in modalità wireless. Questo access point può essere integrato in un router che fornisce la connessione a Internet. A casa vostra, ad esempio, la connessione potrebbe avvenire tramite un **modem**, che può essere un **dispositivo ADSL**, in fibra ottica o, in alcune situazioni, anche tramite connessione satellitare.

Se avete un dispositivo che si connette alla rete Wi-Fi, come un cellulare o un tablet, di solito vi connettete tramite un **router**, che può essere collegato a un modem via **fibra** o ADSL. Anche i servizi come **Starlink**, che offrono **connessione via satellite**, utilizzano un dispositivo simile (uno scatolotto) per la connessione alla rete. L'importante è che tutti questi dispositivi gestiscono solo la **parte fisica** della connessione, ma i **dati** viaggiano nel resto della rete attraverso il **protocollo IP**, indipendentemente dalla tecnologia sottostante (fibra, ADSL, satellite, ecc.).

Sicurezza in Rete: Spam, Sniffing e Cracking

Quando parliamo di **spam**, ci riferiamo a **messaggi indesiderati** che arrivano via **email** o altre forme di comunicazione digitale. Anche se lo spam in sé non è sempre pericoloso, è una risorsa che può essere sfruttata da attaccanti per inviare contenuti dannosi, come link a siti malevoli. **Sniffing**, invece, è l'atto di **intercettare** i pacchetti di dati in transito su una rete. In pratica, un attaccante può "spiare" ciò che un altro utente sta facendo online, raccogliendo **informazioni sensibili**, come le **password**, se non sono protette. Fortunatamente, protocolli come **HTTPS** (che usa **TLS** per crittografare le comunicazioni) proteggono contro questo tipo di attacchi, impedendo che i dati siano facilmente leggibili anche se intercettati.

Il **cracking** è un attacco attivo, che mira a **violare** un sistema per ottenere **accesso non autorizzato**. Questo tipo di attacco è più serio e può portare alla **compromissione di dati sensibili** o all'assunzione di **controllo di un sistema**.

Come Funzionano i Pacchetti e la Diagnostica di Rete

Immaginate di voler inviare un pacchetto di dati a un server in una città distante. I pacchetti di dati viaggiano da un dispositivo all'altro, passando per vari **router** lungo il percorso. Questi router sono dispositivi di rete che si occupano di **"instradare"** i pacchetti verso la destinazione corretta. Se qualcosa va storto e il pacchetto non arriva a destinazione, possiamo utilizzare strumenti diagnostici come **Traceroute** per capire dove si trova il problema.

Traceroute è un programma che invia pacchetti a intervalli e verifica se ciascun router lungo il percorso risponde. Se un router **non risponde**, Traceroute lo **segnala**, permettendoci di individuare esattamente dove si verifica l'interruzione. In pratica, Traceroute ti **mostra il percorso** che il pacchetto segue dalla tua macchina alla destinazione, passo dopo passo, e ti permette di diagnosticare dove si verifica un blocco.

TTL e la Vita dei Pacchetti

Ogni pacchetto che viaggia sulla rete ha un **TTL** (Time-To-Live), che indica il **numero massimo di "salti"** che il pacchetto può fare tra router prima di essere eliminato dalla rete. Questo sistema serve per evitare che un pacchetto resti intrappolato in un **ciclo infinito** a causa di errori nei percorsi di rete, come accade in caso di **loop**.

Ad esempio, se un pacchetto ha un TTL iniziale di 30, significa che può passare attraverso 30 router. Ogni volta che il pacchetto passa un router, il TTL diminuisce di 1. Quando il TTL arriva a 0, il pacchetto viene distrutto dal router, evitando di occupare risorse di rete in modo infinito. Se un pacchetto non arriva a destinazione prima che il TTL arrivi a zero, il router invia un messaggio di errore al mittente, indicando che il pacchetto non è riuscito a raggiungere la destinazione.

Questo meccanismo è molto utile per **evitare congestioni di rete**, specialmente nei casi in cui ci siano errori nell'instradamento dei pacchetti. Se il pacchetto resta in circolazione troppo a lungo senza arrivare a destinazione, il protocollo IP si assicura che venga rimosso.

Diagnostica con Traceroute e PING

Traceroute è uno strumento che permette di diagnosticare dove si trova il problema lungo il percorso di rete. Questo strumento invia pacchetti con TTL decrescente e traccia la rotta che il pacchetto segue fino alla destinazione. Se un router non risponde, Traceroute ci dice esattamente dove si è verificato l'errore. È uno strumento prezioso per comprendere dove si fermano i pacchetti in caso di problemi di connessione.

Un altro strumento molto usato per diagnosticare i problemi di rete è **PING**, che invia pacchetti di prova a una destinazione per verificare se è raggiungibile. **PING** fornisce informazioni sulla **latenza** e sulla **disponibilità della connessione**. Tuttavia, a differenza di **Traceroute**, non fornisce dettagli sul percorso che il pacchetto ha seguito, ma solo se il pacchetto è arrivato a destinazione e quanto tempo ha impiegato.

Il Percorso dei Pacchetti

Ogni pacchetto di dati che viaggia in rete attraversa vari **router**, che sono dispositivi che decidono come instradare i pacchetti tra le diverse reti. Ogni router esamina l'indirizzo di destinazione del pacchetto e decide quale sia la miglior rotta per inviarlo al prossimo router, fino a quando non raggiunge la destinazione finale.

Quando inviamo un pacchetto a un sito web (ad esempio **www.fineco.it**), non è necessario conoscere il percorso esatto che il pacchetto seguirà attraverso la rete. È sufficiente conoscere l'indirizzo IP del sito web, che è risolto tramite il **DNS** (Domain Name System). Il DNS è il sistema che **converte i nomi di dominio** (come **www.fineco.it**) in indirizzi IP, che sono necessari per instradare correttamente i pacchetti.

TTL e Meccanismi di Sicurezza

Il TTL è essenziale per garantire che i pacchetti non rimangano in circolazione indefinitamente, soprattutto in caso di errori nell'instradamento.

Inoltre, **Traceroute** e **PING** sono strumenti utili per diagnosticare le performance della rete, identificare i colli di bottiglia o i router che non rispondono, e per determinare se un pacchetto raggiunge correttamente la destinazione. Entrambi gli strumenti sono basati sul protocollo ICMP (Internet Control Message Protocol), che viene utilizzato per scambiare messaggi di errore e diagnostica tra dispositivi di rete.

Il processo di **comunicazione in rete** si può descrivere come una **trasmissione di dati** tra due o più dispositivi. Questi dispositivi sono collegati in vari modi e comunicano attraverso diverse modalità: **multicast**, **broadcast** e **unicast**. Ogni modalità ha specifiche caratteristiche che determinano come i dati vengono inviati e ricevuti.

Per comprendere come avviene questa comunicazione, ci si affida al **modello OSI** (Open Systems Interconnection), che suddivide il processo di comunicazione in **sette livelli distinti**, ognuno dei quali ha una funzione precisa e interagisce con i livelli adiacenti. I livelli sono disposti in ordine gerarchico, con i dati che viaggiano dal **livello applicativo** fino al **livello fisico** e viceversa.

I livelli del modello OSI:

1. Livello Fisico (Physical Layer)

- **Funzioni:** Definisce le caratteristiche **fisiche** e **elettriche** dei dispositivi di rete. Include specifiche di **tensione**, **velocità di trasmissione dati**, **layout fisico** dei **cavi** e **connettori**, e **sincronizzazione** della trasmissione dei bit.
- **Protocolli e tecnologie:** Ethernet (**fisico**), **DSL**, **USB**, **Bluetooth**, e **IEEE 802.11 (Wi-Fi)** a livello fisico).
- **Elementi di interesse:** **Media di trasmissione** (rame, **fibra ottica**, wireless), dispositivi (**hub**, **ripetitori**, **cavi**), **segnalazione** (modulazione, demodulazione).

2. Livello Data Link (Collegamento Dati)

- **Funzioni:** Responsabile del trasferimento affidabile di **frame** tra nodi direttamente collegati. Gestisce la suddivisione dei dati in **frame**, il **controllo degli errori**, e il **controllo del flusso** tra nodi.
- **Protocolli:** **Ethernet (IEEE 802.3)**, **Wi-Fi (IEEE 802.11)**, **PPP (Point-to-Point Protocol)**, **Frame Relay**, **ATM**.
- **Sottolivelli:**
 - **LLC (Logical Link Control):** Controlla la **sincronizzazione** dei **frame**, la **gestione del flusso**, e il **controllo degli errori**.
 - **MAC (Media Access Control):** Gestisce l'accesso ai **mezzi condivisi**, utilizzando **indirizzi fisici** (indirizzi **MAC**).
- **Elementi di interesse:** **Switch**, **bridge**, **NIC** (schede di rete).

3. Livello Rete (Network Layer)

- **Funzioni:** Fornisce servizi di **instradamento** e **indirizzamento** per la consegna dei **pacchetti** da un nodo sorgente a uno di destinazione, anche su reti diverse. Si occupa anche di **frammentazione** e **riassembaggio** dei **pacchetti**.
- **Protocolli:** **IP (IPv4, IPv6)**, **ICMP (Internet Control Message Protocol)**, **OSPF (Open Shortest Path First)**, **BGP (Border Gateway Protocol)**.
- **Elementi di interesse:** **Router**, **IP addressing**, **subnetting**, protocolli di **routing**.

4. Livello Trasporto (Transport Layer)

- **Funzioni:** Garantisce la consegna affidabile dei dati tra host, offrendo **multiplexing** delle connessioni, **controllo del flusso**, e **gestione degli errori**. Può operare in modalità **orientata alla connessione (TCP)** o **non orientata alla connessione (UDP)**.
- **Protocolli:** **TCP (Transmission Control Protocol)**, **UDP (User Datagram Protocol)**, **SCTP (Stream Control Transmission Protocol)**.
- **Servizi:** Divisione dei dati in **segmenti**, gestione della **sessione** (apertura, mantenimento, chiusura).

5. Livello Sessione (Session Layer)

- **Funzioni:** Gestisce le **sessioni** tra applicazioni, fornendo servizi di apertura, mantenimento, e chiusura delle **connessioni**. Supporta il **dialogo** tra i sistemi, il **controllo dei token**, e la **sincronizzazione** dei dati.
- **Protocolli:** **NetBIOS**, **RPC (Remote Procedure Call)**, **SMB (Server Message Block)**.
- **Elementi di interesse:** **Dialogo a metà-duplex** o **full-duplex**, gestione dei **checkpoint** per riprendere le sessioni interrotte.

6. Livello Presentazione (Presentation Layer)

- **Funzioni:** Converte i dati da un **formato** utilizzato dall'**applicazione** a un **formato standard** per la trasmissione in rete e viceversa. Si occupa anche di **crittografia**, **compressione**, e **traduzione** dei formati di dati.
- **Protocolli:** **SSL/TLS** (per crittografia), **JPEG**, **GIF**, **PNG** (per immagini), **MPEG** (per video).
- **Elementi di interesse:** **Codifica dei caratteri**, conversione dei dati (**EBCDIC** a **ASCII**), **compressione dei dati**.

7. Livello Applicazione (Application Layer)

- **Funzioni:** Fornisce servizi di **rete** agli utenti finali e alle **applicazioni**. Include le **interfacce utente** e i servizi di rete diretti, come l'accesso ai **file**, la **posta elettronica**, e la **navigazione web**.
- **Protocolli:** **HTTP**, **FTP**, **SMTP**, **DNS**, **Telnet**, **SNMP**.
- **Elementi di interesse:** **Browser web**, client di **posta elettronica**, applicazioni di **trasferimento file**.

Interazione tra i Livelli

Ogni livello del modello **OSI** comunica solo con i livelli immediatamente adiacenti. I dati passano dal livello **applicazione** al livello **fisico** sul dispositivo di origine, attraversano il mezzo di comunicazione, e risalgono dal livello **fisico** al livello **applicazione** sul dispositivo di destinazione.

Ogni livello incapsula i dati in unità più piccole: al livello di applicazione si parla di **messaggi**, al livello di trasporto di **segmenti**, al livello di rete di **pacchetti**, e al livello di collegamento di **frame**. Queste unità contengono i **dati** veri e propri, ma anche delle **intestazioni** con informazioni cruciali per la comunicazione, come gli **indirizzi di origine e destinazione**, il **protocollo di comunicazione**, e altre informazioni per la gestione del traffico.

Le **intestazioni** sono fondamentali per il corretto instradamento e la gestione dei pacchetti. Per esempio, nel **livello di rete**, l'intestazione del pacchetto include l'indirizzo **IP di origine e destinazione**, che permette ai pacchetti di arrivare correttamente al loro destinatario. Nel **livello di trasporto**, le intestazioni gestiscono la **sequenza** dei pacchetti e garantiscono che arrivino nell'ordine giusto.

A livello pratico, i **protocolli** di rete, come il già citato **TCP/IP**, giocano un ruolo cruciale nel garantire una **comunicazione corretta**. Ad esempio, TCP garantisce che i pacchetti vengano inviati e ricevuti correttamente, e che eventuali pacchetti persi vengano ritrasmessi. Allo stesso modo, il protocollo **IP** si occupa di determinare come i pacchetti viaggiano attraverso le reti, determinando il **percorso migliore**.

In sintesi, la **comunicazione in rete** si basa su un modello che suddivide il processo in livelli distinti. I **dati** vengono incapsulati e inviati attraverso ciascun livello, con l'ausilio di **intestazioni** che forniscono informazioni fondamentali per il corretto indirizzamento, la gestione degli errori, e il controllo del flusso di comunicazione.

8\10

Una delle domande era: **come viene deciso il numero massimo di salti che può fare un pacchetto e cosa succede se non è abbastanza?**

Il **numero di salti** (TTL, Time To Live) viene deciso dai **dispositivi di rete** o dal **sistema operativo**, ed è solitamente un multiplo di **2**, tra **64** e **128**. Se non è sufficiente, il valore viene **aumentato automaticamente**, soprattutto quando il pacchetto viene scartato troppe volte.

Ora stiamo analizzando il protocollo **TCP/IP**. Questo protocollo ci garantisce che i **pacchetti** arrivino a destinazione, grazie al supporto di dispositivi come il **PCP (Packet Control Protocol)** o il **DTP (Data Transport Protocol)**. Questi protocolli si occupano della gestione della **trasmissione** dei pacchetti e della loro **affidabilità**.

Il **TCP** garantisce che i pacchetti inviati arrivino correttamente al livello **applicativo**.

Ora vediamo alcuni **protocolli applicativi** come **HTTP**, **HTTPS**, e **DNS**.

L'**HTTP** è uno dei protocolli più usati. Quando visitiamo una **pagina web**, il **browser** invia una richiesta **HTTP**, che contiene il **nome dell'host** (tramite **DNS**) e il **percorso** del file richiesto. Il **server web** risponde inviando il **file** richiesto, come un **.gif** o un **.html**. La richiesta **HTTP** viene generalmente fatta sulla **porta 80** del **server**.

Sapete cosa è una **porta**? È un punto di accesso per un **servizio** su una macchina. Ogni **porta** ha un **numero**, come la **porta 80** per **HTTP**, o altre per diversi **servizi**. Il sistema usa questo **numero** per indirizzare correttamente i dati al **processo** giusto.

Ad esempio, quando un **client** fa una richiesta, lo fa su una **porta** del **server**, quando il **server** risponde, usa la propria **porta** per inviare la risposta al **client**.

Il **TCP** gestisce questa comunicazione creando una **connessione** e un **socket** tra il **client** e il **server**. Il **socket** è un'interfaccia per inviare e ricevere **dati**. Quando scrivete un programma in **C**, **Java** o in altri **linguaggi**, dovete utilizzare le **porte** per stabilire una **connessione** e inviare **messaggi**.

L'**HTTP** è un protocollo **stateless**, cioè non ricorda le richieste precedenti. Per mantenere la **sessione** attiva tra il **client** e il **server**, si usano le **sessioni HTTP**, ad esempio in **PHP**..

Le **connessioni HTTP** possono essere **persistenti** o **non persistenti**. La differenza riguarda se la connessione tra **client** e **server** viene mantenuta o chiusa dopo ogni richiesta. Quando un **client** vuole

comunicare con un **server**, stabilisce una **connessione TCP**. Quando il **server** risponde, la **connessione** può rimanere aperta o chiudersi a seconda del tipo di connessione.

Quando il **client** dice "voglio questo file", invia una **richiesta**, e successivamente il **file** viene trasferito. Il file viene trasferito e messo in **esecuzione** a una certa **grandezza**. Per questo motivo, bisogna considerare un **tempo di trasferimento** maggiore, poiché un file di dimensioni più grandi impiegherà più tempo per il **trasferimento**. Non si deve solo considerare il **tempo di invio della richiesta**, ma anche il **tempo di trasferimento del file**, che può essere lungo, come nel caso di **immagini grandi** o **contenuti multimediali**.

Per caricare una **pagina web**, ci sono diverse fasi: la fase di **handshake**, la **richiesta** del file e poi il **tempo di esecuzione** del file. Se voglio ottenere altri file, devo ripetere la stessa procedura per ciascun file, poiché una pagina web contiene più file, come **immagini**, **audio**, **video**, ecc. Per ogni elemento della pagina, devo fare un **handshake** e richiedere ogni file separatamente.

Per ottimizzare, non conviene aprire e chiudere una connessione ogni volta per ogni **file**. È meglio mantenere la **connessione attiva** e trasferire più file in una singola comunicazione. Con il protocollo **HTTP persistente**, la connessione rimane attiva e posso trasferire diversi file senza dover riaprire la connessione per ogni singolo file. Questo riduce il **tempo complessivo** di trasferimento, poiché il server non è in attesa tra una richiesta e l'altra.

Se si inviano più **richieste** insieme, è possibile trasferire più file in un solo ciclo. Questo tipo di comunicazione è definito **HTTP pipeline**. Invece, senza la pipeline, si tratta di **HTTP non-pipeline**, dove i file vengono inviati uno alla volta.

Quando si parla di **messaggi HTTP**, il formato cambia a seconda del livello del protocollo. Per esempio, i **programmi UDP** inviano **segmenti**, mentre a livello **applicativo** si usano i **messaggi HTTP**. I messaggi HTTP sono fatti di **istruzioni di richiesta** e contengono campi come **header** e **valori**. I principali comandi HTTP sono **GET**, **POST**, **PUT**, **DELETE**, ecc. Questi comandi determinano cosa fare con l'oggetto richiesto, come ottenere un file (**GET**), inviare dati (**POST**), modificare un file (**PUT**) o eliminarlo (**DELETE**).

Il **metodo GET** è il più comune e viene utilizzato per ottenere un file o una risorsa dal **server**. Quando usi il metodo **POST**, invece, i **dati** vengono inviati dal **client** al **server**, come nel caso dei **moduli HTML**. Il **POST** è più sicuro del **GET**, poiché i dati non sono visibili nell'**URL** del **browser**. Inoltre, il **POST** permette di inviare grandi quantità di dati nel **corpo** della richiesta, come ad esempio nei **moduli di inserimento**.

Esistono altri metodi come **PUT** e **DELETE**, che vengono utilizzati per **modificare** o **rimuovere** file sul **server**. Questi metodi sono utilizzati soprattutto nelle **API web**, dove si comunica direttamente con il server per modificare oggetti specifici.

Infine, molti **client** e **server** utilizzano il formato **JSON** per scambiare i dati. **JSON** è un formato di scambio di dati molto diffuso che permette di strutturare i dati in modo leggibile e facilmente manipolabile. **Chi di voi conosce già JSON?** Per la prossima lezione, studiate cos'è il formato **JSON**, e uno di voi sarà selezionato per spiegarmelo.

ora parliamo delle prime **cinque fasi** che sono state più approfondite. Queste fasi riguardano un **protocollo** di interazione **client-server** che sfrutta il **protocollo HTTP** per la comunicazione tra client e server. Questo tipo di comunicazione viene spesso implementato utilizzando **REST**. **Chi sa cos'è REST?** Dovete guardare cos'è il protocollo **REST**, che è molto interessante per lo sviluppo di **applicazioni client-server**. **REST** sfrutta **HTTP** per l'interazione con il **server**.

Il **protocollo HTTP** è normalmente usato per ottenere un **file**, come una **pagina web**, e la richiesta che invia il client è tipicamente una **richiesta GET**. Quando si fa una richiesta tramite HTTP, il **client** invia una richiesta a una certa **macchina**. Il pacchetto che arriva contiene una **stringa**, che ha due scopi. Il primo scopo di questa stringa è **indirizzare** il pacchetto alla macchina giusta, simile al comando **PING**, che viene usato per testare la **velocità di comunicazione** tra due dispositivi. Quando invii un comando **PING** a un server, esso risponde per misurare il tempo di **ritorno** dei dati. Per esempio, se fai un ping a **www.utdm.com**, il programma traduce automaticamente l'indirizzo del dominio in un **indirizzo IP** e invia il messaggio a quella macchina.

Inoltre, se usi un servizio di **hosting**, come **Lupa**, che offre **server web**, quando chiedi un **indirizzo IP** per il tuo sito web, il server non ti assegnerà un indirizzo IP unico, ma ti fornirà una macchina che può ospitare **molti siti web**. Quando arriva un messaggio a questa macchina, la macchina deve capire quale **pagina web** deve essere restituita. La macchina lo fa guardando la stringa presente nella richiesta HTTP, che indica il sito web richiesto.

Quando più client fanno richieste allo stesso server, come per esempio **www.leonardo.it**, **www.lucia.it**, e **www.davide.it**, tutti i client inviano la loro richiesta alla stessa macchina. Ma per determinare quale **pagina web** devono ricevere, la macchina esamina la **stringa** presente nel messaggio per indirizzare la richiesta al sito giusto. Ad esempio, quando un **browser** invia una richiesta al server per **www.terminato.it/index.html**, il server esamina la stringa per determinare quale **pagina web** restituire. Se la stringa indica un percorso come **/index.html**, il server va a prendere il file **index.html** dalla cartella corretta del **file system**.

In sostanza, il server web ascolta sulla porta **80** e, quando riceve una richiesta con una stringa, guarda il percorso specificato per restituire la pagina web giusta, come **index.html**, dalla directory corrispondente nel file system.

In generale, se vuoi creare un **sito web**, il tuo server avrà un **indirizzo IP pubblico**. Il server web può ospitare più di un sito, e ogni sito è identificato tramite la **stringa** che viene inviata nella richiesta HTTP. Questi indirizzi pubblici sono gestiti da **hosting provider**, che ti forniscono l'accesso ai server per ospitare il tuo sito web.

Quando si utilizza un **URL**, come **www.sanscool.eu-sondev-pick.json**, questa stringa viene tradotta nel **DTP (Data Transfer Protocol)**, che è un protocollo **HTTP**. Il **messaggio** viene inviato sempre dalla stessa **macchina** e **porta**, perché si utilizza lo stesso **protocollo HTTP**. Lato **server**, la stringa non viene solo utilizzata per identificare gli **indirizzi**, ma viene anche inserita come **campo** nel **messaggio** stesso. Per questo motivo, è necessario recuperare il file **samdir-page.html** da un **percorso** preciso dell'**hard disk**.

Un aspetto da considerare riguarda i **percorsi**. In alcuni casi, è possibile utilizzare la notazione **../** per risalire di una **directory** nel **file system**. Per esempio, se nel percorso è scritto **../**, si risale di un livello, mentre **../..** significa risalire di due livelli, e così via. Senza **controlli adeguati**, è possibile accedere a **directory non autorizzate**. Questo problema è stato fortunatamente ridotto, poiché molti **server web** limitano l'accesso alle sole **partizioni designate**, impedendo l'accesso all'intero **disco**.

Inoltre, alcuni campi nell'**header** della richiesta **HTTP** sono utilizzati per identificare il tipo di **client**, come il **user-agent**, che indica il tipo di **browser** utilizzato. Questo è importante perché il **server** potrebbe rispondere in modo diverso a seconda del **browser**. Il campo **Accept-Language** indica anche la **lingua preferita** dal **client**, il che permette al **server** di rispondere con la **versione corretta** del **sito web** in base alla **lingua** selezionata. Se la **lingua** richiesta non è disponibile, il **server** invierà il contenuto nella **lingua predefinita**.

Nel **messaggio HTTP** ci sono diversi altri campi, come la **lunghezza del corpo** del **messaggio** e il **tipo di contenuto**, che può essere, per esempio, **text/html** per una **pagina web** o **image/jpeg** per un'immagine. Questi dettagli sono importanti per la corretta **visualizzazione** del contenuto.

Ogni richiesta **HTTP** inviata dal **client** al **server** include anche una serie di **codici di risposta**. Ad esempio, il codice **200** indica che la richiesta è stata elaborata correttamente, mentre il **404** indica che la **pagina** non è stata trovata. Ci sono anche codici come **301** (pagina spostata) e **505** (versione non supportata).

Il **server** risponde al **client** con i **dati** richiesti, come un **file HTML** o un'immagine. In caso di un **errore**, il **server** invierà un **codice di errore** insieme a un **messaggio** che spiega il problema. È anche possibile che il **server** risponda con una versione in **cache** del **file** richiesto se la versione originale non è stata aggiornata di recente.

Un altro aspetto importante è che **HTTP** permette di inviare **messaggi di testo** che possono essere anche creati manualmente, senza bisogno di un **browser**. Ad esempio, utilizzando una **sessione remota**, è possibile inviare direttamente una **richiesta HTTP** al **server**, digitando il comando **GET** seguito dall'**URL** del **file** desiderato.

Quando si interagisce con il **server**, la **risposta** può essere visualizzata come **testo semplice**, senza l'utilizzo di un **browser grafico**. Questo processo può essere utile per comprendere meglio il funzionamento dei **server web** e delle **richieste HTTP**.

Per esempio, in alcune situazioni si deve essere particolarmente **fortunati** per ottenere un risultato desiderato. Un esempio riguarda l'"**elasticato**" e i "**puntetti**" da 40, che non sono lanciati in **TPS**, ma comunque si tratta di un caso in cui si verifica un **lancio** in **TPS**.

Questa prova può essere utile per vedere come si comporta il **sistema**.

Quando ci si connette a un **server**, quasi tutti i **server** moderni utilizzano **HTTPS**, mentre quelli più vecchi rispondono ancora in **HTTP**. È importante comprendere che se ci si connette a un **server** che non è configurato per **HTTPS**, il **browser** potrebbe non supportare adeguatamente la connessione sicura. Per questo motivo, quando si scrive un'URL nel **browser**, è fondamentale specificare il **protocollo**. Ad esempio, bisogna scrivere **http://** o **https://**, a seconda della configurazione del **server**.

Un aspetto interessante da considerare è che, se il **server web** è configurato per rispondere sia in **HTTP** che in **HTTPS**, e si fa una richiesta **HTTP**, il **server** potrebbe automaticamente reindirizzare la connessione a **HTTPS**, garantendo così una maggiore **sicurezza**.

Nel contesto di **server web** e **HTTP**, è essenziale anche comprendere i concetti di **cookie** e di come vengano utilizzati per **memorizzare** le preferenze dell'utente, come ad esempio la sezione di un **sito web** che l'utente visita più frequentemente. In questo caso, il **server** memorizza le informazioni relative all'utente nei **cookie** per inviarlo direttamente alla **sessione** desiderata al prossimo accesso.

La gestione dei **cookie** è un aspetto importante della navigazione web, e se il **server** sa che un utente preferisce una determinata sezione del sito, lo **indirizza** direttamente lì, senza bisogno di ulteriori richieste. I **cookie** vengono memorizzati sul dispositivo dell'utente, ma è possibile anche eliminarli. La gestione dei **cookie** può variare in base alle preferenze dell'utente, e alcuni **siti web** permettono di scegliere se accettarli o meno.

Il trattamento dei **dati** personali tramite i **cookie** è una pratica che riguarda la **profilazione** dell'utente. Profilare significa raccogliere informazioni sull'utente per personalizzare l'esperienza di navigazione, ma è anche importante sapere che l'utente ha il diritto di sapere come vengono gestiti i suoi dati. Quando un **sito web** raccoglie informazioni tramite i **cookie**, dovrebbe informare l'utente in modo trasparente, ad esempio con un messaggio che spiega che i **cookie** vengono utilizzati per migliorare l'esperienza.

Un aspetto fondamentale nella gestione dei **cookie** è la possibilità per l'utente di cancellarli. La domanda che sorge è se sia possibile vedere o cancellare i **cookie** memorizzati nel proprio dispositivo. La risposta è sì: tramite le **impostazioni** del **browser**, l'utente può visualizzare e cancellare i **cookie**. Inoltre, i **cookie** possono essere utilizzati dal **server** per memorizzare informazioni specifiche su di noi, come ad esempio le sezioni più visitate di un **sito web**.

Per concludere, i **cookie** sono fondamentali per la personalizzazione dell'esperienza web, ma è importante che gli utenti siano consapevoli di come vengono raccolti e trattati i loro **dati** personali.

Quello che stiamo trattando riguarda vari aspetti della **sicurezza informatica**, delle **sessioni web** e delle **tecniche di ottimizzazione** come la **cache** e i **proxy**.

Ad esempio, quando un utente accede a un sito web, il server riconosce l'utente tramite un **ID univoco** (ad esempio, un numero come "8734") che viene usato per personalizzare l'esperienza dell'utente. Questo meccanismo è simile a come funziona quando un utente compra un nuovo computer e sceglie di trasferire i dati dal vecchio dispositivo. Se il sito ha già registrato un **profilo utente**, la sessione può essere ripristinata, permettendo all'utente di continuare da dove aveva interrotto.

Quando si parla di **sessioni web** e **cookie**, è importante capire che i cookie sono file di **memoria temporanea** che memorizzano dati relativi alla navigazione dell'utente, come il suo comportamento di acquisto o le preferenze di navigazione. I **cookie** possono essere utilizzati per personalizzare l'esperienza di navigazione, ma possono anche rappresentare un rischio di **sicurezza**, se non gestiti correttamente, dando la possibilità a malintenzionati di **manipolare la sessione** dell'utente.

Per quanto riguarda la **criptografia**, se i server utilizzano tecniche di crittografia, è importante che questi siano configurati correttamente per evitare che gli **attacchi di decrittazione** compromettano la sicurezza. Gli **esperti di crittografia** devono progettare algoritmi robusti e metodi sicuri per cifrare i dati sensibili. Se questi non vengono gestiti correttamente, ad esempio attraverso l'uso di chiavi di **criptografia** deboli o vulnerabili, i dati possono essere **decifrati** da attaccanti.

La **cache** è un meccanismo utilizzato per **velocizzare l'accesso ai dati**. In informatica, la **cache** memorizza informazioni frequentemente richieste, come le pagine web o le risposte a richieste HTTP, così che possano essere recuperate rapidamente senza dover rifare tutta la richiesta al server. Esistono diverse forme di cache, tra cui quelle **lato client** (memorizzate nel browser dell'utente) e quelle **lato server** (memorizzate su un server proxy che risponde alle richieste degli utenti).

Un **proxy** è un server che **intercetta le richieste** tra il client (utente) e il server web, memorizzando una copia dei contenuti richiesti. Se più utenti fanno la stessa richiesta (ad esempio, per visualizzare la stessa pagina web), il proxy fornirà una **copia locale** della pagina, riducendo il **traffico di rete** e migliorando la velocità di caricamento. I **proxy** possono essere utilizzati anche per motivi di **sicurezza**, per **nascondere l'identità** dell'utente o per **filtrare** il traffico.

L'utilizzo della **cache** e dei **proxy** è particolarmente utile per ridurre il **traffico di rete** e migliorare l'esperienza dell'utente, specialmente quando si accede a contenuti statici (come immagini o pagine HTML che non cambiano frequentemente). Per le **pagine dinamiche**, invece, la cache può essere meno efficace, perché le informazioni possono cambiare frequentemente.

Quando si parla di **protocollo HTTP**, è importante sapere che si basa su **TCP** (Transmission Control Protocol), che garantisce una **trasmissione affidabile** delle informazioni tra client e server. Tuttavia, **HTTP** non include un vero e proprio **livello di sessione** come in altri modelli di comunicazione, come **FTP** (File Transfer Protocol). FTP, infatti, utilizza **due porte** (20 e 21) per separare la **trasmissione dei file** dalla **gestione dei comandi**, garantendo una **connessione stabile** durante il trasferimento.

La comprensione di questi concetti è essenziale per affrontare le sfide legate alla **sicurezza web** e alla **gestione delle sessioni**. Inoltre, è fondamentale sapere che molti **attacchi** possono essere effettuati sfruttando le vulnerabilità nei **cookie**, nella gestione delle **sessioni** o nelle **cache**.

9\10

La posta elettronica si basa su tre protocolli principali: SMTP, POP e IMAP, ognuno dei quali svolge un ruolo specifico nel processo di invio e ricezione dei messaggi.

1. **SMTP (Simple Mail Transfer Protocol)**: Utilizzato per inviare le email. Funziona sulla porta 25, anche se oggi spesso vengono usate porte alternative (come la 587 per motivi di sicurezza). SMTP si occupa di trasferire i messaggi dalla casella di posta in uscita del mittente al server del destinatario, che poi li recapitano alla casella del destinatario.
2. **POP (Post Office Protocol)**: Questo protocollo è utilizzato per scaricare i messaggi dal server di posta sul client (ad esempio sul computer o sul telefono). Una volta che il messaggio viene scaricato, viene solitamente rimosso dal server, il che significa che il messaggio è accessibile solo sul dispositivo da cui è stato scaricato.
3. **IMAP (Internet Message Access Protocol)**: A differenza di POP, IMAP permette di mantenere i messaggi sul server. Questo consente di accedere alla posta elettronica da più dispositivi (ad esempio telefono, computer, tablet) e di mantenere tutto sincronizzato, poiché i messaggi non vengono eliminati dal server ma rimangono disponibili per la consultazione da qualsiasi dispositivo.

Come funziona la posta elettronica?

Quando invii un'email, essa viene inviata dal tuo client tramite il server di posta, che la smista fino al destinatario, passando da uno o più server. Se stai inviando un messaggio a qualcuno nel tuo stesso dominio (per esempio, se usi lo stesso provider di posta elettronica), il messaggio passa attraverso un solo server di posta. Se invece il destinatario usa un altro dominio, il messaggio verrà inoltrato attraverso più server fino ad arrivare a destinazione.

Un aspetto interessante è che, anche se il server di posta si trova a distanza (per esempio in un altro paese), il protocollo TCP garantisce il trasporto "magico" del messaggio senza la necessità di passare per decine di server. Ad esempio, se stai inviando un'email a qualcuno in Australia, il messaggio potrebbe passare per server intermedi, ma il protocollo TCP si occupa di garantire che arrivi a destinazione nel modo più efficiente possibile.

Inoltre, la posta elettronica standard trasferisce solo testo (in formato ASCII). Tuttavia, grazie all'estensione **MIME (Multipurpose Internet Mail Extensions)**, è possibile inviare anche file multimediali come immagini, video e audio, che vengono comunque convertiti in formato testo (per esempio in base64) per essere trasmessi correttamente.

Configurazione del client di posta

Per utilizzare la posta elettronica, è necessario configurare correttamente il client (ad esempio Outlook, Thunderbird, o l'app di posta sul telefono). Durante la configurazione, devi inserire alcuni dati, come il server di posta in entrata (POP o IMAP) e il server di posta in uscita (SMTP), nonché le credenziali per autenticarti. Questi dati permettono al client di comunicare con il server e di inviare o ricevere correttamente i messaggi.

In sintesi:

- **SMTP** si occupa di inviare email.
- **POP** scarica email dal server.
- **IMAP** mantiene le email sul server per l'accesso da più dispositivi.

Tutti questi protocolli sono essenziali per il corretto funzionamento della posta elettronica, permettendo la comunicazione tra mittenti e destinatari in modo efficiente e sicuro.

Motivi per cui l'e-mail potrebbe non essere arrivata:

1. L'indirizzo e-mail potrebbe non esistere (es. **dmi.unibg.it**).
2. Il sistema potrebbe non essere funzionante correttamente (anche se, in realtà, non è questo il caso).
3. La posta potrebbe essere ancora in coda da inviare, soprattutto se ci sono molte e-mail in attesa.
4. Normalmente, il tempo di invio di una e-mail è di 30 secondi, quindi non dovrebbe esserci un lungo ritardo.
5. Se non si hanno i permessi per accedere al sistema (ad esempio, **Pippo**), l'invio dell'e-mail non è possibile.

Sicurezza e problemi di invio:

- Se non si è autenticati correttamente, non si può inviare un messaggio a nome di un'altra persona (es. Marika deve autenticarsi prima di inviare un messaggio a nome suo).
- Per evitare lo **spam**, i server di posta sono configurati per impedire l'uso da parte di utenti non autorizzati. Ciò avviene tramite **autenticazione** o considerando come **fidata** una rete interna (come quella dell'università).
- I server di posta non fanno un **relay** per tutti; possono inviare posta solo agli utenti autorizzati o all'interno della rete interna dell'istituto.

Tecniche di sicurezza:

- Le principali regole della sicurezza sono espresse nella sigla **CIA**: **Confidenzialità**, **Integrità**, e **Disponibilità**. Questi sono i tre obiettivi fondamentali della sicurezza nelle comunicazioni.

Conclusione:

- La configurazione di un sistema di posta prevede che solo gli utenti autenticati possano inviare e-mail, evitando quindi l'uso non autorizzato (ad esempio, un attacco di **denial of service** tramite invio di posta non autenticata).

La sicurezza nelle comunicazioni si basa su tre principi fondamentali, riassunti nell'acronimo **CIA**: **Confidenzialità, Integrità e Disponibilità**. Questi principi assicurano che le informazioni siano protette da accessi non autorizzati, mantengano la loro veridicità e siano sempre disponibili quando necessario. Questi aspetti, però, non includono **autenticazione**.

L'autenticazione è un processo separato, ma ugualmente cruciale. Quando usiamo un servizio online, prima dobbiamo essere **autenticati**, cioè il sistema deve verificare chi siamo. Dopo di che, il sistema decide se possiamo o meno eseguire determinate azioni: è qui che entra in gioco l'**autorizzazione**. Infine, l'**accounting** si riferisce alla registrazione di tutte le azioni che compiamo, in modo che nessuno possa negare di averle eseguite. Questo è particolarmente utile per motivi di sicurezza, perché, ad esempio, se un utente invia una e-mail, questa azione viene registrata e, in caso di necessità, può essere tracciata.

Quando si parla di **server di posta**, spesso si utilizza **Telnet** per connettersi al server, generalmente sulla porta **25**, che è quella standard per l'invio delle e-mail. Tuttavia, sarebbe possibile connettersi anche ad altre porte (come la **porta 26**), ma non è detto che si ottenga lo stesso risultato, a meno che la configurazione del server non sia stata cambiata appositamente.

Un tema molto rilevante nella sicurezza informatica è il **phishing**. Si tratta di una truffa in cui un attaccante cerca di ingannare l'utente facendosi passare per una fonte affidabile, come una banca o un ente pubblico, per ottenere informazioni personali o credenziali sensibili. Esistono diversi tipi di phishing, tra cui il **phishing tradizionale**, dove l'attaccante invia un messaggio con allegati dannosi, e il **spear phishing**, in cui l'attaccante si finge una persona di fiducia, magari per indurre un pagamento falso.

Riconoscere un messaggio di phishing non è sempre facile, ma ci sono alcuni segnali a cui prestare attenzione. Ad esempio, gli indirizzi e-mail spesso sono leggermente modificati per sembrare legittimi, ma un occhio attento può notare dettagli come lettere che appaiono simili, ma con leggere differenze, come nel caso della "A" con il puntino sopra. Inoltre, molti messaggi di phishing contengono link che rimandano a siti web falsi, progettati per rubare informazioni.

Infine, è importante sapere che i sistemi di posta elettronica sono configurati per prevenire l'invio di e-mail non autorizzate, come nel caso dello **spam**. Per evitare abusi, i server di posta permettono solo a utenti **autenticati** di inviare messaggi e tracciano tutte le azioni effettuate. Se una persona invia e-mail a nome di qualcun altro senza essere autorizzata, come nel caso di un attacco di **denial of service** o spam, è possibile risalire all'autore dell'invio grazie ai sistemi di **accounting**.

Durante questa lezione, ci siamo focalizzati su diversi aspetti legati alla sicurezza informatica, alla gestione della privacy e alla protezione dei dati personali. Un tema centrale è stato come le vulnerabilità nei sistemi possano essere sfruttate dagli attaccanti, spesso attraverso strumenti come i Trojan, ma anche come la normativa, come il GDPR, cerchi di proteggere gli utenti e i loro dati.

Abbiamo iniziato parlando di **Trojan**, che sono programmi dannosi creati per entrare nel sistema di un utente e dare il controllo remoto all'attaccante. Un esempio che è stato discusso riguarda la creazione di un **video compromettente** utilizzando una webcam, ottenuto proprio grazie a un Trojan. La minaccia si manifesta quando l'attaccante può ricattare la vittima, chiedendo, per esempio, un pagamento in **bitcoin** per evitare che il video venga diffuso. Questi tipi di attacchi sono tra i più comuni nella cybercriminalità, poiché sfruttano la paura e il senso di vulnerabilità della vittima. L'idea di essere "controllati" e minacciati può spingere a pagare anche somme elevate.

Un altro argomento fondamentale trattato è stato la gestione dei **dati personali**. Quando ci registriamo su un sito web, forniamo informazioni che potrebbero finire esposte se il sito subisce una violazione. In particolare, il concetto di **DataBridge** è stato introdotto come un esempio di servizio che consente di verificare se la propria email è stata compromessa in qualche data breach. DataBridge è utile perché permette agli utenti di sapere se i propri dati sono finiti in mano sbagliata, cosa che può aiutare a prevenire

danni più gravi. A volte, però, accade che i dati vengano esposti senza che l'utente ne sia consapevole, perché certi siti non gestiscono adeguatamente la sicurezza.

Abbiamo poi discusso della normativa **GDPR** e dei diritti degli utenti riguardo ai propri dati. Il GDPR è una legge che si occupa di proteggere la privacy degli utenti e assicura che i loro dati vengano trattati correttamente. Un diritto fondamentale che gli utenti hanno è quello di chiedere la **cancellazione** dei propri dati personali dai siti web, ma questo diritto non sempre viene rispettato. La situazione è piuttosto complessa: mentre alcuni servizi, come Facebook, permettono di eliminare l'account e i dati ad esso collegati, altri, come McDonald's, potrebbero non avere politiche altrettanto chiare o facili da seguire.

Un altro aspetto interessante riguarda le **email di phishing**. Si è parlato di come gli attaccanti sfruttano l'inganno per ottenere informazioni sensibili, come password o dettagli bancari, cercando di convincere le vittime a cliccare su link malevoli o ad aprire allegati pericolosi. A volte, i tentativi di phishing possono sembrare incredibilmente credibili, come se provenissero da fonti ufficiali, rendendo difficile per l'utente medio distinguerli da comunicazioni legittime.

Nel corso della lezione, abbiamo anche esaminato la **sicurezza dei server di posta elettronica**, in particolare il funzionamento del protocollo **SMTP**. Questo protocollo è utilizzato per inviare messaggi di posta da un client a un server, ma la sicurezza delle email dipende molto dal tipo di infrastruttura utilizzata. Gli attacchi possono avvenire se il server di posta non è configurato correttamente o se non vengono utilizzati strumenti di protezione come **antivirus** o **antispam**. Inoltre, strumenti di sicurezza come **DKIM (DomainKeys Identified Mail)** vengono utilizzati per verificare che un'email provenga effettivamente dal dominio dichiarato nel mittente. È un modo per assicurarsi che l'email sia autentica e non sia stata manipolata da un attaccante.

Un punto importante emerso è come **le email possano passare attraverso più server** durante il loro percorso. Ogni server che gestisce una mail può aggiungere delle intestazioni che consentono di tracciare la sua provenienza e verificarne l'integrità. Gli amministratori di sistema possono usare questi dati per capire se l'email è stata compromessa o se proviene effettivamente dalla fonte legittima. Questo è particolarmente utile per rilevare attività sospette e fermare i tentativi di phishing o altre minacce.

Inoltre, durante la lezione, abbiamo parlato di come gli **antivirus** e gli **anti-spam** svolgano un ruolo fondamentale nella protezione delle email, monitorando e filtrando i messaggi per identificare virus o contenuti dannosi. Questi sistemi sono spesso il primo ostacolo che un attaccante deve superare per riuscire a infettare un dispositivo.

Infine, il tema delle **connessioni sicure** è stato affrontato, in particolare parlando dei protocolli utilizzati per connettersi ai servizi di posta elettronica. Abbiamo esplorato come i client di posta si connettono ai server di Gmail, ad esempio, tramite porte specifiche (come la porta 443 per una connessione HTTPS sicura) e come questa connessione sicura aiuti a proteggere i dati durante il loro trasferimento.

Durante la lezione, si è parlato di vari aspetti tecnici legati alla gestione di server web, client di posta elettronica, e la configurazione delle porte di rete. La discussione è iniziata con un chiarimento su alcune porte di rete utilizzate dai protocolli di comunicazione. La porta 80 è associata ai server web che utilizzano **HTTP** (HyperText Transfer Protocol), mentre la porta 443 è utilizzata da **HTTPS** (HTTP sicuro), che è una versione crittografata di HTTP. Quindi, quando ci si connette a un sito web, è importante notare se la connessione è sicura o meno, con HTTPS che protegge la comunicazione tra il client (per esempio, il browser) e il server web.

Tuttavia, si è evidenziato che se ci si connette a un server di posta elettronica, ad esempio per inviare una mail tramite **SMTP** (Simple Mail Transfer Protocol), la porta comunemente utilizzata è la **porta 25**. Quindi, se si sta cercando di inviare una email, la porta 25 è quella giusta. Invece, se il server di posta è configurato per l'invio sicuro, potrebbe essere utilizzata una porta diversa, come la porta 587, per garantire una comunicazione cifrata. La discussione ha anche trattato del protocollo di **posta elettronica IMAP e POP**, che permettono di leggere i messaggi dalla casella di posta.

Un altro punto interessante della discussione è stato l'uso del file **hosts**. In un contesto di debugging, il file hosts è molto utile perché permette di associare manualmente un nome a un indirizzo IP. In pratica, se uno sviluppatore sta lavorando su un sito web e vuole testarlo prima di renderlo pubblico, può scrivere nel file hosts il nome del dominio (come www.miosito.it) e associarlo all'indirizzo IP del server che ospita il sito.

Questo metodo consente di bypassare il sistema DNS (Domain Name System) e di testare il sito localmente prima che il nome del dominio venga propagato su Internet.

Per esempio, se un server web ha più di un sito web (un "server virtuale"), è essenziale che il browser sappia su quale sito si sta tentando di connettersi. Questo è possibile grazie al file hosts o, se il sito è già online, attraverso il sistema DNS. Quando si scrive un indirizzo web nel browser, come `www.miosito.it`, il DNS risolve il nome in un indirizzo IP, che consente al computer di connettersi al server corretto. Se, però, si scrive direttamente un indirizzo IP nel browser, il server deve essere in grado di capire quale sito ospita, in caso ci siano più siti sulla stessa macchina, proprio come nel caso del file hosts.

In sintesi, la discussione ha coperto vari aspetti della connessione a internet, dai protocolli di posta (SMTP, IMAP, POP3) alle configurazioni dei server web, passando per l'importanza dei file di configurazione come **hosts** per il debugging e per la gestione dei siti web. Questi concetti sono cruciali per comprendere come funziona internet sotto il cofano e come la comunicazione tra client e server avviene in modo sicuro ed efficiente.

15\10

La crittografia viene utilizzata per proteggere i dati, garantendo la loro **confidenzialità** (solo le persone autorizzate possono leggere le informazioni), **integrità** (le informazioni non sono alterate) e **disponibilità** (le informazioni sono sempre accessibili quando necessario).

Crittografia Simmetrica

La **crittografia simmetrica** è uno dei metodi più antichi e utilizzati. In questo caso, sia per cifrare che per decifrare i dati, viene utilizzata la stessa chiave. Un esempio semplice di crittografia simmetrica è il **cifrario di Cesare**, in cui ogni lettera del testo in chiaro viene spostata di un numero fisso di posizioni nell'alfabeto. Ad esempio, con uno shift di 3, la lettera 'A' diventa 'D', la 'B' diventa 'E', e così via.

Questo metodo, pur essendo facile da comprendere, è molto vulnerabile agli attacchi, come il **brute force**, dove un attaccante prova tutte le possibili combinazioni di chiavi (nel caso di un cifrario di Cesare, solo 25 possibili spostamenti).

Crittografia con Cifrario di Vigenère

Un altro esempio di cifrario simmetrico è il **Cifrario di Vigenère**, che è più complesso e sicuro rispetto al cifrario di Cesare. In questo caso, viene utilizzata una chiave che viene ripetuta per tutta la lunghezza del testo. La cifra è ottenuta dalla somma delle posizioni delle lettere del testo e della chiave. Sebbene più sicuro, anche il Cifrario di Vigenère può essere vulnerabile a determinati attacchi se la chiave è debole o ripetuta.

Crittanalisi

La **criptanalisi** è la scienza che studia come decifrare un messaggio senza conoscere la chiave segreta. Gli attaccanti cercano di sfruttare vulnerabilità nell'algoritmo di cifratura per trovare la chiave o il testo in chiaro.

I tipi di attacchi includono:

- **Ciphertext-only attack:** L'attaccante ha solo il testo cifrato e deve cercare di scoprire la chiave.
- **Known-plaintext attack:** L'attaccante conosce sia il testo in chiaro che il testo cifrato corrispondente.
- **Chosen-plaintext attack:** L'attaccante ha il potere di scegliere il testo in chiaro da cifrare e analizzare il risultato.

Sicurezza Computazionale

La **sicurezza computazionale** di un algoritmo di cifratura dipende da due fattori:

1. **Il costo per rompere il cifrario** deve essere maggiore rispetto al valore dell'informazione cifrata.

2. **Il tempo necessario per decifrare il messaggio** deve essere lungo abbastanza da rendere l'attacco impraticabile.

La crittografia moderna si basa su algoritmi più avanzati che utilizzano chiavi molto lunghe e complessi algoritmi matematici, rendendo gli attacchi come il brute force estremamente difficili da realizzare.

In generale, la crittografia è uno strumento fondamentale per la sicurezza delle comunicazioni in rete e per la protezione dei dati sensibili.

Il contesto della crittografia simmetrica e l'evoluzione dal DES al Triple DES (3DES) e AES

Nel contesto della **crittografia simmetrica**, la protezione dei dati attraverso algoritmi che utilizzano una **chiave segreta condivisa** per cifrare e decifrare il contenuto è un concetto fondamentale. Il **DES (Data Encryption Standard)** è stato uno degli algoritmi più utilizzati per la cifratura dei dati sin dagli anni '70. Ideato per soddisfare le esigenze di sicurezza dell'epoca, il DES utilizza una **chiave di 56 bit** per cifrare blocchi di **64 bit** di dati. Tuttavia, con il progresso tecnologico e l'aumento della potenza di calcolo, il DES è diventato vulnerabile a **attacchi di forza bruta**, poiché la lunghezza della chiave di 56 bit non è sufficiente a garantire una protezione robusta.

Il DES e la sua vulnerabilità

Il **DES** è stato progettato per un'epoca in cui le risorse computazionali erano limitate, ma con il passare degli anni e l'aumento della capacità dei computer, l'algoritmo ha mostrato i suoi limiti. La lunghezza della chiave (56 bit) ha reso possibile a un attaccante di testare rapidamente tutte le combinazioni possibili attraverso tecniche di forza bruta. Questo ha portato alla necessità di trovare soluzioni alternative che permettessero di aumentare la sicurezza mantenendo un sistema efficiente.

La soluzione con Triple DES (3DES)

Un passo fondamentale verso l'evoluzione del DES è stato il **Triple DES (3DES)**, che utilizza **tre chiavi di 56 bit** ciascuna, estendendo la chiave complessiva a **168 bit**. Questo ha reso gli attacchi di forza bruta molto più difficili, ma l'algoritmo, pur migliorando la sicurezza, ha una grande limitazione: la **velocità**. Poiché i dati vengono cifrati tre volte (una volta per ogni chiave), l'algoritmo diventa notevolmente più lento rispetto al DES originale, soprattutto quando si trattano grandi volumi di dati.

Funzionamento del Triple DES

Il funzionamento del Triple DES si basa su una sequenza di operazioni cifratura-decifratura-cifratura:

1. **Cifratura:** Il testo in chiaro (plaintext) viene cifrato utilizzando il primo algoritmo DES e la chiave **K1**.
2. **Decifratura:** Il risultato della cifratura precedente viene decifrato con una seconda chiave **K2** utilizzando l'algoritmo DES, e questa operazione è l'inverso della cifratura.
3. **Cifratura finale:** Il risultato della decifratura viene quindi cifrato di nuovo con una terza chiave **K3**, producendo il **testo cifrato finale**.

Questa sequenza di operazioni fornisce maggiore sicurezza rispetto al DES, ma la **velocità** dell'algoritmo è penalizzata dal fatto che vengono eseguiti tre passaggi di cifratura. Per migliorare ulteriormente la sicurezza, si possono utilizzare due o tre chiavi distinte, con il risultato che **la chiave complessiva può arrivare a 168 bit**, migliorando notevolmente la sicurezza rispetto al DES originario.

L'evoluzione: AES (Advanced Encryption Standard)

Nel 1997, a causa delle limitazioni del Triple DES, il **NIST (National Institute of Standards and Technology)** ha avviato una competizione per sviluppare un nuovo standard di crittografia che fosse più sicuro e più efficiente. Dopo anni di selezione, l'algoritmo **Rijndael** è stato scelto come il nuovo **Advanced Encryption Standard (AES)** nel 2001.

Il principale vantaggio dell'AES rispetto al DES e al Triple DES è la sua **velocità** e la sua capacità di utilizzare chiavi di **128, 192 e 256 bit**, a seconda del livello di sicurezza desiderato. AES è progettato per

operare su blocchi di **128 bit** di dati, ed è in grado di gestire sia chiavi corte che lunghe in modo efficiente, rendendolo molto più veloce e sicuro rispetto ai suoi predecessori.

Struttura di AES

Il processo di cifratura in AES è organizzato in una serie di **round**. A differenza del DES, che si basa su una serie di **operazioni di permutazione e sostituzione**, AES utilizza una struttura diversa, che include i seguenti passaggi:

- 1. **Sostituzione dei byte**: Ogni byte del blocco di dati viene sostituito con un altro byte usando una **tabella S-box**.
- 2. **Shift delle righe**: Le righe della matrice vengono spostate ciclicamente per ottenere maggiore diffusione.
- 3. **Mix delle colonne**: Le colonne della matrice vengono combinate per migliorare la diffusione dei dati.
- 4. **Aggiunta della chiave**: Una chiave derivata dalla chiave principale viene aggiunta all'output del round, per fornire una protezione aggiuntiva.

L'algoritmo AES utilizza un numero variabile di **round**, che dipende dalla lunghezza della chiave: **10 round** per una chiave da **128 bit**, **12 round** per una chiave da **192 bit** e **14 round** per una chiave da **256 bit**.

La fase di **decifratura** è l'inverso della cifratura, ma con l'ordine dei passaggi invertito.

Confronto tra DES, Triple DES e AES

Il DES e il Triple DES sono stati fondamentali nella storia della crittografia simmetrica, ma oggi sono considerati obsoleti a causa delle loro limitazioni in termini di sicurezza e velocità. L'AES, invece, offre un'architettura più moderna, con una maggiore sicurezza e una velocità di esecuzione significativamente più alta, ed è il nuovo standard per la crittografia simmetrica.

Ecco un confronto tra i tre algoritmi:

Caratteristica	DES	Triple DES	AES
Lunghezza della chiave	56 bit	112 o 168 bit	128, 192, 256 bit
Numero di round	16	48 (3x16)	10, 12 o 14
Velocità	Elevata, ma obsoleta	Lenta	Alta
Sicurezza	Bassa (vulnerabile)	Migliorata	Molto alta
Uso attuale	Obsoleto	Obsoleto	Standard moderno

Conclusioni

Il **DES**, pur essendo stato una pietra miliare nella storia della crittografia, non è più considerato sicuro a causa della vulnerabilità delle chiavi brevi. Il **Triple DES** è stato una soluzione temporanea, ma la sua **lentezza** lo rende poco pratico. Con l'introduzione di **AES**, un algoritmo più sicuro, efficiente e versatile, il mondo della crittografia ha fatto un grande passo avanti. AES rimane oggi il **gold standard** per la crittografia simmetrica, proteggendo i dati in modo robusto e garantendo alte prestazioni.

Numeri Random e Pseudo-Random

I numeri random sono numeri che non possono essere predetti e non seguono alcun schema deterministico. La loro imprevedibilità è fondamentale in vari contesti, come la generazione delle chiavi per algoritmi crittografici. Un esempio è l'algoritmo **RSA**, che è basato sulla **crittografia a chiave pubblica/privata** e richiede numeri random per la generazione delle chiavi. Inoltre, i numeri random vengono utilizzati per la creazione di **chiavi temporanee** (session key) in vari algoritmi simmetrici e per la **distribuzione delle chiavi**, come nel caso di **Kerberos**.

Tuttavia, nella pratica, spesso non si utilizzano veri e propri numeri random, ma **numeri pseudo-random**. Un numero pseudo-random è generato da un algoritmo deterministico, che, dato lo stesso input (chiamato **seed**), produce sempre lo stesso output. Sebbene non siano completamente casuali, la loro sequenza è sufficientemente difficile da prevedere senza conoscere il seed.

Criteri per numeri random

Per essere considerata **random**, una sequenza di numeri deve soddisfare alcune condizioni:

- **Uniformità**: la distribuzione dei numeri (ad esempio, 1 e 0 in una sequenza binaria) deve essere uniforme, ossia il numero di 1 e 0 deve essere approssimativamente uguale.
- **Indipendenza**: non deve essere possibile inferire una parte della sequenza conoscendo un'altra parte. In altre parole, una sottosequenza non deve essere predicibile.

Generazione dei Numeri Random

Esistono diverse modalità per generare numeri random o pseudo-random:

- **True Random Number Generator (TRNG)**: utilizza una sorgente fisica di entropia (ad esempio, il movimento del mouse o il rumore termico di un circuito elettronico) per produrre numeri veramente casuali. Questi numeri sono considerati **veramente random**, poiché dipendono da fenomeni fisici imprevedibili.
- **Pseudo-Random Number Generator (PRNG)**: un algoritmo deterministico che genera sequenze di numeri che sembrano casuali, ma che sono completamente determinati da un **seed** iniziale. Un esempio di questo tipo di generatore è la **Pseudo-Random Function (PRF)**, utilizzata nella generazione di chiavi simmetriche.

Nel contesto della crittografia, la sequenza di numeri deve essere **imprevedibile** per evitare che un attaccante possa predire i numeri successivi in una sequenza, anche se conosce alcuni numeri precedenti.

RSA e Crittografia a Chiave Pubblica/Privata

L'algoritmo **RSA** è un esempio di crittografia **asimmetrica**, dove sono utilizzate due chiavi: una **pubblica** (per cifrare i dati) e una **privata** (per decifrare i dati). La sicurezza di RSA si basa sulla difficoltà di fattorizzare numeri molto grandi, in particolare il prodotto di due numeri primi scelti casualmente.

I numeri random sono essenziali nella generazione delle chiavi RSA. La generazione delle chiavi in RSA coinvolge la scelta di numeri primi grandi e l'uso di un algoritmo matematico deterministico che, dato il primo numero, produce la chiave privata associata alla chiave pubblica.

Crittografia Simmetrica e Asimmetrica

Nel contesto della **crittografia simmetrica**, viene utilizzata una **chiave segreta condivisa** tra i partecipanti per cifrare e decifrare i messaggi. In contrasto, la **crittografia asimmetrica** (come RSA) usa una coppia di chiavi pubblica/privata.

La **cifratura a flusso** è un tipo di crittografia simmetrica in cui i dati vengono cifrati bit per bit (o byte per byte), piuttosto che in blocchi di dati. In questo tipo di cifratura, viene generata una **chiave di flusso** che viene combinata con il testo in chiaro per ottenere il testo cifrato. L'algoritmo **RC4** è un esempio di cifratura a flusso.

La **chiave di sessione** è una chiave simmetrica utilizzata per cifrare la comunicazione tra due parti per una determinata sessione. La sua generazione e distribuzione sicura sono fondamentali in molti protocolli, come **Kerberos**.

Autenticazione del Messaggio

Un altro aspetto importante della sicurezza è l'**autenticazione del messaggio**. Anche se la cifratura simmetrica può garantire la confidenzialità di un messaggio, non fornisce alcuna garanzia sull'autenticità dell'origine del messaggio. In altre parole, senza un meccanismo di autenticazione, un attaccante potrebbe intercettare il messaggio e modificarlo senza essere rilevato.

L'**autenticazione del messaggio** è quindi necessaria per garantire che il messaggio provenga da una fonte legittima e che non sia stato alterato. Un modo comune per garantire questa autenticazione è l'uso del **Message Authentication Code (MAC)**, che viene generato utilizzando una **chiave segreta condivisa** tra il mittente e il destinatario.

Come funziona il Message Authentication Code (MAC)

Il **MAC** è calcolato applicando una funzione di hash crittografica o un algoritmo di cifratura simmetrica al messaggio, combinato con una chiave segreta. Il destinatario, avendo la stessa chiave segreta, può calcolare il MAC dal messaggio ricevuto e verificarne l'integrità confrontandolo con il MAC inviato dal mittente. Se i due MAC coincidono, il messaggio è autentico e non è stato alterato.

Esempio di MAC:

1. Il mittente **A** calcola il MAC utilizzando il messaggio e la chiave segreta.
2. Il mittente invia il messaggio e il MAC a **B**.
3. Il destinatario **B** calcola un nuovo MAC dal messaggio ricevuto e verifica se corrisponde a quello inviato.
4. Se i MAC coincidono, il messaggio è autentico; altrimenti, potrebbe esserci stato un errore o una manipolazione.

In sintesi, i numeri random e pseudo-random sono essenziali per garantire la sicurezza in vari algoritmi crittografici, inclusi quelli utilizzati nella **crittografia a chiave pubblica** come RSA e in quelli a chiave simmetrica come i cifrari a flusso. Inoltre, mentre la cifratura garantisce la confidenzialità, l'**autenticazione del messaggio** è cruciale per evitare attacchi come la falsificazione o la manipolazione dei dati. I **Message Authentication Code (MAC)**, che utilizzano chiavi segrete, sono uno strumento efficace per garantire l'integrità e l'autenticità del messaggio, completando così la protezione dei dati in transito.

Message Authentication Code (MAC)

Il **Message Authentication Code (MAC)** viene inviato insieme al messaggio al ricevente. L'interlocutore riceve sia il messaggio che il MAC. Per verificarne l'integrità, l'interlocutore calcola il MAC usando la **chiave segreta condivisa** e il messaggio ricevuto. Se il MAC calcolato coincide con quello ricevuto, il destinatario è sicuro che il messaggio non è stato alterato e che proviene dalla persona giusta.

Funzioni Hash

Una **funzione hash** è una funzione che prende in input un dato di qualsiasi dimensione e restituisce una stringa di una lunghezza fissa. Ad esempio, **SHA-256** produce sempre un output di 256 bit, indipendentemente dalle dimensioni dell'input. Le funzioni hash sono utilizzate per garantire l'integrità dei dati: se un messaggio viene modificato, anche l'hash cambierà, rendendo chiaro che il messaggio è stato alterato.

Perché una funzione hash sia utile per l'autenticazione, deve possedere alcune caratteristiche:

1. **One-way**: è praticamente impossibile risalire all'input originale dato l'hash.
2. **Collision-resistant**: è computazionalmente difficile trovare due input diversi che producono lo stesso hash.
3. **Strong Collision Resistance**: è difficile trovare due messaggi distinti che abbiano lo stesso hash.

Ad esempio, **SHA-1** non è più sicuro a causa delle collisioni trovate, dove due messaggi distinti producono lo stesso hash.

Le funzioni hash sono utilizzate anche per verificare che i documenti scaricati siano genuini. Se l'hash di un file scaricato corrisponde all'hash fornito dal sito web, il file è integro. Se l'hash è diverso, il file potrebbe essere stato manomesso.

MAC con Crittografia Simmetrica

In crittografia simmetrica, sia il mittente che il destinatario condividono una chiave segreta. Il processo di autenticazione prevede che il mittente calcoli l'hash del messaggio e lo cifri con la chiave segreta. Il messaggio cifrato e l'hash vengono inviati al destinatario. Il destinatario utilizza la stessa chiave per decifrare l'hash e confrontarlo con l'hash calcolato dal messaggio ricevuto. Se i due hash corrispondono, il messaggio è autentico.

MAC con Crittografia Asimmetrica

Nella **crittografia asimmetrica**, vengono utilizzate due chiavi: una **pubblica** e una **privata**. La chiave pubblica è conosciuta da tutti, mentre la chiave privata è solo del destinatario. Un esempio di crittografia asimmetrica è l'algoritmo **RSA**.

In questo caso, il mittente utilizza la sua **chiave privata** per cifrare l'hash del messaggio. Quando il destinatario riceve il messaggio, utilizza la chiave pubblica del mittente per decifrare l'hash e verificare l'autenticità del messaggio.

Autenticazione con Hash e Valori Segreti

In alcuni casi, invece di condividere una chiave segreta, i due interlocutori possono condividere un valore segreto **S**. Questo valore viene concatenato al messaggio, e l'hash del messaggio e del valore segreto viene inviato. Il destinatario esegue lo stesso calcolo per verificare che l'hash corrisponda. Se è corretto, sa che il messaggio proviene dalla persona giusta.

La Sicurezza della Chiave e la Necessità di un Canale Sicuro

Sia nelle comunicazioni con **crittografia simmetrica** che **asimmetrica**, è fondamentale che la chiave (se condivisa) o il valore segreto rimangano **segreti**. Se una chiave o un valore segreto viene scoperto, un attaccante potrebbe facilmente decifrare i messaggi. Per questo motivo, le chiavi segrete e i valori condivisi devono essere trasmessi tramite canali sicuri.

Applicazioni delle Funzioni Hash

Le funzioni hash sono ampiamente utilizzate in vari scenari:

- **Verifica dell'integrità dei file**: Ad esempio, quando scarichi un file da internet, il suo hash può essere verificato per accertarsi che non sia stato manomesso.

- **Gestione delle password:** Le password non sono memorizzate come testo chiaro, ma come hash. Quando un utente inserisce la password, viene confrontato l'hash.
- **Intrusion Detection Systems:** In alcuni sistemi di sicurezza, l'hash dei file viene memorizzato per identificare eventuali modifiche non autorizzate.

La Criptografia Asimmetrica

La **criptografia asimmetrica** è un tipo di crittografia che utilizza una **coppia di chiavi**: una **pubblica** e una **privata**. Ogni utente ha una chiave pubblica che può essere condivisa con chiunque, mentre la chiave privata viene tenuta segreta.

La crittografia asimmetrica è utilizzata in vari ambiti, come:

- **Firma digitale:** Un messaggio cifrato con la chiave privata può essere verificato da chiunque con la chiave pubblica dell'autore del messaggio.
- **Distribuzione delle chiavi:** La crittografia asimmetrica è anche utile per la distribuzione sicura delle chiavi simmetriche, utilizzando la chiave pubblica per cifrare la chiave simmetrica.

RSA e Altri Algoritmi

RSA è uno degli algoritmi di crittografia asimmetrica più noti. Utilizza la chiave pubblica per cifrare il messaggio e la chiave privata per decifrarlo. La sicurezza di RSA si basa sulla difficoltà di fattorizzare numeri molto grandi.

In un esempio pratico:

- **Alice** usa la chiave pubblica di **Bob** per cifrare il messaggio.
- **Bob** usa la sua chiave privata per decifrare il messaggio ricevuto.

La crittografia asimmetrica consente la **riservatezza** dei messaggi e la **verifica dell'autenticità** tramite le firme digitali, che sono una versione digitale delle firme tradizionali.

In sintesi, i **MAC** e le **funzioni hash** sono strumenti fondamentali per garantire l'integrità e l'autenticità dei messaggi in vari sistemi crittografici. La **criptografia simmetrica** utilizza una chiave condivisa per cifrare e decifrare i messaggi, mentre la **criptografia asimmetrica** impiega una coppia di chiavi pubblica e privata, aumentando la sicurezza e la flessibilità nelle comunicazioni.

Rispetto alla crittografia asimmetrica, quando volete inviare un messaggio a qualcuno, prendete la **chiave pubblica** di quella persona, cifrate il messaggio e lo inviate. In questo caso, sapete che solo quella persona può decifrarlo, dato che solo lei possiede la **chiave privata**. Se invece vi interessa **validare la firma** di quella persona (verificare che sia davvero lei a inviare il messaggio), usate la sua **chiave pubblica**. Questo processo è opposto rispetto alla cifratura del messaggio.

Questa procedura permette di garantire l'**autenticità** del messaggio, cioè che provenga realmente dalla persona che dice di essere, e che il messaggio non sia stato modificato.

Perché si applica l'**hash** del messaggio, piuttosto che firmare il messaggio intero? L'**hash** (ad esempio **SHA-256**) fornisce un'uscita di dimensione fissa, che assicura l'integrità del dato. Firmare solo l'hash invece che il messaggio intero è anche più efficiente, soprattutto se il messaggio è molto grande, poiché firmare l'intero messaggio sarebbe più lento e computazionalmente più costoso.

I protocolli di rete regolano il funzionamento di internet, consentendo l'interazione tra server, client e applicazioni. Tra i protocolli fondamentali troviamo SMTP, POP e IMAP, usati rispettivamente per l'invio e la gestione della posta elettronica. Il protocollo **SMTP (Simple Mail Transfer Protocol)**, ad esempio, viene utilizzato per inviare messaggi e si collega alla porta 25 del server destinatario.

Gestione della posta elettronica

Quando inviamo un messaggio, il server SMTP del mittente si collega a quello del destinatario e inserisce il messaggio in una coda. Se leggiamo la posta tramite un client web (come Gmail o Outlook), il server utilizza protocolli come **POP (Post Office Protocol)** o **IMAP (Internet Message Access Protocol)** per rendere disponibili i messaggi.

- **POP:** Scarica i messaggi sul dispositivo e li rimuove dal server.
- **IMAP:** Lascia una copia dei messaggi sul server, permettendo la sincronizzazione su più dispositivi.

L'IMAP è particolarmente utile perché consente di accedere alla posta da diversi dispositivi senza duplicati. Per quanto riguarda la sicurezza, l'autenticazione è essenziale: senza di essa, i server non consentono di inviare email, evitando così lo **spam**.

Codifica dei contenuti non testuali

Quando inviamo contenuti come immagini, audio o video via email, questi vengono convertiti in formato testuale utilizzando lo standard MIME (Multipurpose Internet Mail Extensions). Questo garantisce che i file possano essere trasmessi attraverso protocolli originariamente pensati per il solo testo.

Il DNS e la risoluzione dei nomi

Il **DNS (Domain Name System)** traduce i nomi di dominio (es. www.google.com) in indirizzi IP utilizzabili dai dispositivi. Questo sistema è organizzato gerarchicamente:

1. **Root server:** Gestisce le richieste iniziali.
2. **Server TLD (Top-Level Domain):** Gestisce domini come .com, .it, ecc.
3. **Server autorevoli:** Forniscono informazioni sui domini specifici (es. google.com).

Funzionamento del DNS

Quando un utente digita un URL nel browser, il dispositivo invia una richiesta al DNS configurato. Se il server locale non ha già l'indirizzo IP richiesto, inoltra la richiesta ai server di livello superiore fino a ottenere una risposta.

I record DNS contengono informazioni chiave:

- **Record A:** Associa un nome di dominio a un indirizzo IP.
- **Record MX:** Specifica i server di posta per un dominio.

Un esempio pratico: se si desidera inviare una mail al dominio mpg.it, il client di posta chiede al DNS il record MX corrispondente e ottiene l'indirizzo del server SMTP per inoltrare il messaggio.

Sicurezza del DNS

Un attacco noto come **DNS Poisoning** altera le informazioni dei server DNS, reindirizzando gli utenti verso siti fraudolenti. Questo tipo di attacco può essere usato per rubare credenziali, come le password dei conti bancari. Per contrastare tali minacce, sono stati sviluppati protocolli come **DNSSEC (DNS Security Extensions)**, che aggiungono firme digitali ai record DNS.

Il ruolo dei firewall

Un **firewall** è un sistema di sicurezza che filtra il traffico in base a criteri predefiniti, come indirizzi IP di origine e destinazione, porte e protocolli. I firewall possono:

- Bloccare accessi non autorizzati.
- Consentire il traffico solo da fonti fidate.
- Impedire che dati sensibili vengano trasmessi a server esterni senza autorizzazione.

Questi strumenti e protocolli sono essenziali per garantire il corretto funzionamento e la sicurezza delle comunicazioni in rete. Dai protocolli di posta come SMTP e IMAP al DNS e ai firewall, ogni componente contribuisce a creare un ecosistema digitale stabile e sicuro.

Differenze tra TCP e UDP

Il protocollo **TCP** (Transmission Control Protocol) e il **UDP** (User Datagram Protocol) garantiscono una comunicazione efficace, semplificando il lavoro a livello applicativo: l'applicazione non deve preoccuparsi di ciò che avviene nei livelli sottostanti. Tuttavia, esistono delle **differenze fondamentali** tra i due protocolli.

Prima di iniziare a trasmettere un pacchetto, in TCP si esegue una fase di **handshaking** per concordare vari parametri di comunicazione, come la velocità o le dimensioni dei pacchetti. UDP, invece, "va e basta", risultando più rapido ma meno affidabile, poiché non garantisce servizi aggiuntivi come il controllo di flusso o la gestione della congestione.

Controllo di flusso e congestione

Nel controllo di flusso, i dispositivi regolano la velocità di trasmissione: se una macchina ricevente è lenta, la trasmittente adegua il ritmo, evitando di inviare troppi pacchetti. Quando ciò non avviene, i pacchetti in eccesso vengono scartati, causando ritrasmissioni e incrementando il traffico complessivo.

Il **controllo di congestione**, invece, si occupa di regolare il traffico complessivo: se la rete è congestionata, si riduce l'emissione dei pacchetti per alleggerire il carico.

Funzioni di multiplexing e de-multiplexing

I protocolli di trasporto, come TCP e UDP, svolgono funzioni di **multiplexing** e **de-multiplexing**. Questi processi permettono di indirizzare correttamente i dati:

- Il multiplexing consente di inviare pacchetti da più applicazioni verso il livello di rete.
- Il de-multiplexing consente di ricevere pacchetti dalla rete e indirizzarli all'applicazione corretta, specificando l'indirizzo di destinazione e la porta.

Problemi legati alle dimensioni dei pacchetti

A livello fisico, esistono limiti alla grandezza dei pacchetti. Quando un messaggio viene trasmesso dall'applicazione al livello di trasporto, esso viene suddiviso in segmenti più piccoli (spacchettamento). Ogni segmento riceve un'**intestazione** (header) specifica per il livello corrispondente.

Ad esempio, un'immagine scaricata da una pagina web subisce questo processo:

1. Il messaggio applicativo riceve l'intestazione del livello applicativo.
2. Quando scende al livello di trasporto, si aggiunge l'intestazione di trasporto.
3. Lo stesso avviene per gli altri livelli fino al livello fisico.

Questo processo garantisce che ogni segmento possa essere elaborato correttamente quando arriva a destinazione.

Utilizzo delle porte

Le porte identificano univocamente i processi coinvolti nella comunicazione. Per esempio, supponiamo che il processo P2 voglia connettersi al server P3 tramite TCP:

- La porta sorgente potrebbe essere 9157.

- La porta destinazione potrebbe essere 6428 (la porta del server).

Quando il server risponde, i ruoli delle porte si invertono:

- La porta sorgente diventa 6428.
- La porta destinazione torna ad essere 9157.

Questo sistema consente di gestire più connessioni contemporaneamente, associando ogni connessione a coppie univoche di porte sorgente e destinazione.

Velocità e affidabilità

Se non è necessario il controllo del traffico o delle connessioni, UDP è più veloce poiché non utilizza l'**handshake** e non verifica la ricezione dei pacchetti. Tuttavia, questa rapidità può comportare la perdita di dati in caso di congestione o errori di rete.

Checksum e domande finali

Il checksum è un meccanismo che verifica l'integrità dei dati. Se i dati ricevuti non corrispondono al checksum calcolato, significa che sono stati corrotti durante la trasmissione.

Domande del professore e risposte:

- **Domanda:** Cos'è l'handshaking e come influisce sulla velocità di trasmissione dei dati?
 - **Risposta:** L'**handshaking** è il processo iniziale in cui due dispositivi stabiliscono i parametri di comunicazione (come la velocità e la dimensione dei pacchetti). Questo processo aumenta l'affidabilità della connessione, ma può rallentare la trasmissione rispetto a UDP, che non richiede questa fase preliminare.
- **Domanda:** Qual è la differenza tra il controllo di flusso e il controllo di congestione?
 - **Risposta:** Il **controllo di flusso** regola la velocità di trasmissione per evitare che il ricevente venga sopraffatto, mentre il **controllo di congestione** regola la quantità complessiva di traffico sulla rete per evitare il sovraccarico della rete stessa.
- **Domanda:** Che cos'è il multiplexing e come viene utilizzato nel protocollo TCP?
 - **Risposta:** Il **multiplexing** è il processo che consente di inviare pacchetti da più applicazioni al livello di rete, utilizzando una singola connessione di rete. In TCP, questo permette di gestire più comunicazioni simultanee tra dispositivi.
- **Domanda:** Perché i pacchetti vengono suddivisi in segmenti più piccoli e quali sono le implicazioni per la trasmissione?
 - **Risposta:** I pacchetti vengono suddivisi in segmenti più piccoli per garantire che possano essere inviati in modo efficiente attraverso la rete. La divisione in segmenti riduce il rischio di perdita di dati e permette la gestione di pacchetti di dimensioni compatibili con i limiti fisici delle reti.
- **Domanda:** Qual è la funzione della porta in una connessione TCP?
 - **Risposta:** La **porta** serve a identificare univocamente i processi coinvolti in una comunicazione. Ogni connessione TCP è associata a una coppia di porte (sorgente e destinazione), che consente di indirizzare correttamente i dati tra le applicazioni.

22/10

Il lavoro sulle reti e il loro funzionamento

Iniziamo a parlare delle **reti** e di come esse funzionano, in particolare per quanto riguarda i **protocolli di trasporto** come il **TCP** e l'**UDP**. Quando parliamo di **reti wireless**, si fa riferimento a tecnologie che consentono la comunicazione senza fili, e ci si chiede come queste reti possano garantire l'integrità e la tempestività dei messaggi.

Il **protocollo TCP** è progettato per garantire che i dati arrivino a destinazione in modo corretto, anche se ci possono essere dei difetti lungo il percorso. Al contrario, l'**UDP** è utilizzato per applicazioni in cui è importante che i dati arrivino rapidamente, come ad esempio nella visione di film in streaming, dove la perdita di qualche frame non compromette l'esperienza complessiva.

A livello di applicazioni, i protocolli **HTTP**, **SMTP**, e altri utilizzano il protocollo **TCP** per inviare pacchetti di dati. Questi pacchetti contengono informazioni non solo sul corpo del messaggio, ma anche su chi lo invia e chi lo riceve. Nel protocollo TCP, ogni pacchetto è identificato da un numero di sequenza che permette di rimettere in ordine i dati, dato che i pacchetti potrebbero arrivare in tempi diversi a causa di congestioni o altre interferenze nella rete.

Il segmento TCP e i suoi campi

Un **segmento TCP**, che è una sorta di "messaggio" di trasporto, contiene alcuni campi fondamentali, come la **porta sorgente** e la **porta di destinazione**. È importante che ogni pacchetto di dati contenga queste informazioni per garantire che il messaggio arrivi al destinatario giusto. Inoltre, nel segmento TCP c'è un campo opzionale che indica la lunghezza del segmento, poiché in un protocollo a lunghezza variabile la dimensione dei dati può cambiare.

Il protocollo TCP si occupa anche del **multiplexing**, che consiste nell'invio di più messaggi attraverso la stessa connessione. Questo è importante perché, a livello fisico, esiste una lunghezza massima per i pacchetti che possono essere inviati, quindi un grande messaggio potrebbe essere suddiviso in più pacchetti. Il numero di sequenza serve a rimettere in ordine questi pacchetti quando arrivano a destinazione.

Controllo di flusso e gestione della finestra

Un altro concetto fondamentale in TCP è il **controllo del flusso**, che gestisce la quantità di dati che vengono inviati. Ad esempio, in un meccanismo di **finestra scorrevole**, il mittente invia più pacchetti di dati e aspetta una conferma da parte del destinatario. Se la finestra è grande, il mittente può inviare più pacchetti senza aspettare la conferma di ogni singolo pacchetto. Tuttavia, se la finestra è piccola, il mittente deve aspettare una conferma per ogni pacchetto inviato. La gestione della finestra dipende dalla congestione della rete e dalla capacità di gestione del destinatario.

Se un pacchetto non viene ricevuto o viene perso, la finestra si riduce per evitare che troppi pacchetti vengano inviati troppo velocemente e che la rete diventi **congestionata**. Questo algoritmo di gestione della finestra permette di regolare l'invio dei pacchetti per evitare che i dati si accumulino eccessivamente. La finestra di congestione si adatta dinamicamente per garantire un flusso di dati ottimale.

Reti IP e indirizzamento

A livello del protocollo **IP**, l'indirizzamento è cruciale per il corretto instradamento dei pacchetti. Ogni pacchetto di dati, chiamato **datagramma**, include informazioni sull'indirizzo di destinazione e altre informazioni come il **tempo di vita** (TTL). Il tempo di vita indica quante volte un pacchetto può essere inoltrato attraverso un router prima che venga scartato, evitando che i pacchetti restino bloccati in loop infiniti in caso di errori di instradamento.

Instradamento dinamico

L'**instradamento dinamico** è il processo attraverso il quale i **router** decidono come inviare i pacchetti verso la destinazione. I router utilizzano algoritmi di **routing** per determinare il percorso migliore, che può cambiare in base alla congestione o a guasti nella rete. Questo viene fatto attraverso tabelle di routing che contengono informazioni sui percorsi e le distanze tra i nodi della rete.

Conclusione

Questa panoramica ci ha mostrato come il protocollo **TCP**, attraverso la gestione della finestra e il controllo del flusso, garantisce che i pacchetti di dati vengano inviati in modo sicuro e ordinato. D'altra parte, il protocollo **IP** si occupa di instradare i pacchetti verso la destinazione giusta, utilizzando informazioni come l'indirizzo di destinazione e il tempo di vita del pacchetto.

Gli **indirizzi IP** sono scritti in un formato che consiste in 4 numeri separati da punti, con ogni numero compreso tra 0 e 255. Per esempio, l'indirizzo 154.321.27.3 è un **indirizzo IP** che può identificare una macchina specifica. Un errore potrebbe essere il numero 321, che non è valido, poiché ogni numero deve essere compreso tra 0 e 255.

Ad esempio, l'indirizzo 154.48.27.3 identifica una macchina in rete. Se esistesse un'altra macchina con l'indirizzo 154.48.27.4, le due macchine sarebbero **vicine**, poiché condividono la stessa parte iniziale dell'indirizzo. Questo è simile a come si leggono gli indirizzi di residenza, dove tutte le macchine di una certa università potrebbero avere un prefisso comune, come tutte le macchine italiane che potrebbero iniziare con 154.

In un **sistema di routing**, la **tabella di routing** contiene informazioni su come instradare i pacchetti. Ogni **router** guarda solo una parte dell'indirizzo **IP** per decidere dove inviare i pacchetti, ad esempio i primi tre numeri dell'indirizzo IP. Così, se un pacchetto è destinato all'indirizzo 154.48.27.3 o 154.48.27.4, non cambia nulla per il router che utilizza il primo segmento dell'indirizzo per determinare la strada da percorrere. Saranno comunque inviati alla stessa destinazione iniziale, anche se il router finale dovrà poi decidere se il pacchetto deve essere inviato al 27.3 o al 27.4.

Le **tabelle di routing** permettono ai **router** di instradare i pacchetti senza conoscere l'intero indirizzo IP. Ad esempio, tutti gli indirizzi da 154.1.0.0 a 154.100.0.0 potrebbero essere instradati verso una determinata destinazione, semplificando la gestione degli indirizzi IP. Se dovessero esserci troppi indirizzi da gestire, la tabella di routing diventerebbe troppo grande e complessa.

Ogni **router** ha una "vista" ridotta degli indirizzi IP, in base alla sua posizione nella rete. Quando un pacchetto viaggia attraverso una rete, i **router** lungo il cammino utilizzano solo una parte dell'indirizzo **IP** per instradare il pacchetto correttamente. Se voglio indirizzare un pacchetto solo a una macchina specifica, guardo la parte dell'indirizzo che mi interessa, mentre se voglio includere una vasta gamma di indirizzi, posso usare una **maschera di sottorete**.

La **maschera di sottorete** (subnetting) è una tecnica che consente di dividere una rete in sotto-reti più piccole. Se voglio solo un gruppo di indirizzi IP che appartengono a una determinata rete, applico una **maschera di rete**, che limita la parte dell'indirizzo da usare per fare l'instradamento. Questo consente ai **router** di gestire più facilmente la distribuzione dei pacchetti.

La **conversione** tra numeri decimali e binari è un processo fondamentale per capire come funzionano gli indirizzi **IP**. Ad esempio, il numero 154 in decimale viene convertito in binario come 10011010, e ogni byte dell'indirizzo **IP** è rappresentato da 8 bit (un byte). Quando si fa **subnetting**, la "maschera di rete" determina quale parte dell'indirizzo è dedicata alla rete e quale parte è disponibile per l'host.

Le **tabelle di routing** possono anche cambiare dinamicamente a seconda delle condizioni della rete. Se un **router** diventa non disponibile, la rete potrebbe ricalcolare le rotte migliori, utilizzando **protocolli di routing dinamico** come **RIP** o **OSPF**. Questi protocolli consentono ai **router** di adattarsi ai cambiamenti nella rete senza bisogno di intervento manuale.

Domande e Risposte

1. **Come funzionano le reti wireless e quali sono le principali tecnologie coinvolte nella loro comunicazione?**
 - Le **reti wireless** funzionano utilizzando onde radio per trasmettere i dati, eliminando la necessità di connessioni fisiche. Le principali tecnologie includono **Wi-Fi**, che è ampiamente utilizzato per connettere dispositivi a Internet in ambienti locali, e **Bluetooth**, usato per connessioni a corto raggio tra dispositivi. Altri standard come **5G** offrono velocità più elevate e copertura più vasta per la connessione a dispositivi mobili.
2. **Qual è la differenza tra TCP e UDP e in quali scenari vengono utilizzati?**

- **TCP** garantisce la consegna sicura dei dati, con meccanismi di controllo come il **ritrasmissione** dei pacchetti persi e la gestione dell'ordine dei pacchetti, ed è usato per applicazioni che richiedono affidabilità, come la **navigazione web** (HTTP) o la **posta elettronica** (SMTP). **UDP**, al contrario, non fornisce garanzie sulla consegna, ed è più veloce per applicazioni che tollerano la perdita di pacchetti, come la **videochiamata** o il **streaming di contenuti**.
3. **Come viene gestito il numero di sequenza in TCP e quale ruolo ha nel processo di trasmissione?**
- Il **numero di sequenza** in TCP è utilizzato per identificare e ordinare correttamente i pacchetti. Ogni pacchetto inviato ha un numero di sequenza unico che aiuta il destinatario a riassemblare i pacchetti nell'ordine corretto, anche se arrivano fuori ordine a causa di ritardi nella rete. Se un pacchetto non arriva, il numero di sequenza consente al destinatario di richiedere il **ritrasferimento**.
4. **Quali sono i principali campi di un segmento TCP e quale funzione svolgono?**
- Un **segmento TCP** include vari campi come:
 - **Porta sorgente e porta di destinazione**: identificano il mittente e il destinatario.
 - **Numero di sequenza**: per ordinare i pacchetti.
 - **Numero di conferma**: per confermare la ricezione dei pacchetti.
 - **Lunghezza del segmento**: che indica la dimensione del pacchetto.
 - **Flag**: come SYN e ACK, che controllano l'inizio e la fine delle comunicazioni.
5. **Cos'è il multiplexing in TCP e come viene gestito il flusso dei dati?**
- Il **multiplexing** è un processo che consente di inviare più flussi di dati attraverso una sola connessione TCP. Questo viene fatto attraverso l'uso di **porte** per distinguere tra diverse comunicazioni. Ogni applicazione che invia dati su TCP è associata a una **porta** specifica, e il sistema può inviare più messaggi simultaneamente su una connessione senza che si verifichino conflitti.
6. **Cos'è il controllo di flusso in TCP e come funziona la finestra scorrevole?**
- Il **controllo di flusso** regola la quantità di dati che il mittente può inviare prima di ricevere una conferma dal destinatario. Il **meccanismo della finestra scorrevole** consente al mittente di inviare più pacchetti, ma solo una quantità che il destinatario può gestire. La finestra si sposta in avanti man mano che i pacchetti vengono confermati dal destinatario.
7. **Come viene gestita la congestione in TCP e come influisce sulla finestra di congestione?**
- La congestione viene gestita dinamicamente utilizzando la **finestra di congestione**, che regola la quantità di dati inviati. Se i pacchetti vengono persi o il destinatario è sopraffatto, la finestra viene ridotta per evitare il sovraccarico della rete. La finestra cresce lentamente quando

23/10

Concetti di base su indirizzi IP e rete

Si inizia parlando degli **indirizzi IP** e della necessità di configurare correttamente le reti per evitare conflitti. Gli **indirizzi IP** sono numeri utilizzati per identificare dispositivi in una rete, e questi devono essere **contigui** all'interno di una rete per essere gestiti correttamente. Ogni dispositivo di rete ha un indirizzo IP univoco, e ogni **interfaccia di rete** su un dispositivo, come un **router**, ha il proprio indirizzo. Ad esempio, un router con tre interfacce avrà tre indirizzi IP distinti.

Strutturazione degli indirizzi IP

Si discute anche della suddivisione degli indirizzi IP in **classi** e del concetto di **maschere di rete**. Le **maschere di rete** sono utilizzate per determinare quante **sottoreti** possono essere create all'interno di una rete più ampia. Il numero di **bit** disponibili per gli indirizzi varia a seconda della **maschera**: ad esempio, una **maschera di rete 24** (notata come /24) consente di avere un certo numero di indirizzi disponibili per i dispositivi nella rete.

Si introduce anche il concetto di **sottorete** e di **indirizzo di broadcast**, e viene spiegato che ogni rete ha un **indirizzo di rete** e un **indirizzo di broadcast**. Ad esempio, in una rete con la **maschera /28**, si avrà un numero limitato di indirizzi disponibili per gli **host**, che possono essere utilizzati da dispositivi come computer o server. Gli indirizzi di rete e broadcast sono riservati e non possono essere utilizzati per gli **host**.

Gestione dei datagrammi e instradamento

Si passa poi alla discussione su come i **datagrammi** vengono inviati attraverso una rete. Ogni **datagramma** contiene l'**IP sorgente** e l'**IP di destinazione**, e viene instradato verso la sua destinazione tramite i **router**, che utilizzano tabelle di **instradamento** per decidere quale percorso seguire. Il protocollo **IP** è responsabile per il trasporto dei pacchetti di dati attraverso la rete.

Perdita dei datagrammi e trasporto

Un altro punto discusso riguarda la possibilità di **perdere un datagramma** durante il viaggio attraverso la rete. Se un datagramma viene perso, il protocollo di **trasporto** si occupa di gestire la sua **riemissione**. I datagrammi vengono assemblati in **segmenti** di trasporto che vengono successivamente trasformati in **messaggi** inviati alle **applicazioni**.

Considerazioni sulle classi di indirizzi IP

L'insegnante continua a parlare delle **classi di indirizzi IP** e della loro gestione. Le **classi A, B, e C** erano usate in passato per organizzare gli indirizzi, ma oggi si utilizza principalmente il concetto di **maschera di rete**. Si fa riferimento anche alle **classi private** come il blocco **172.x.x.x**, che non è visibile su Internet, e alle **reti pubbliche**.

Conclusioni sui concetti trattati

Infine, si discute di come configurare correttamente una rete in base alla **maschera di rete** e di come determinare gli **indirizzi validi** in una determinata rete. Si sottolinea che l'utilizzo della **maschera** è fondamentale per capire quali indirizzi sono disponibili per gli **host** e quali sono riservati.

Abbiamo capito come funziona l'**instradamento** tramite gli **indirizzi IP**. Se una macchina non ha un indirizzo IP, non può essere instradata da nessuna parte. Ecco perché i **pacchetti** a livello **IP**, chiamati **datagrammi**, devono includere l'**IP sorgente** e l'**IP di destinazione**, altrimenti non possono raggiungere la loro destinazione. Quando una macchina vuole comunicare con un'altra, a livello **applicativo**, invia un pacchetto tramite il **protocollo IP**. Il pacchetto viene instradato attraverso vari dispositivi di rete fino a raggiungere la destinazione.

La domanda è: come fa una macchina a sapere se un'altra macchina si trova nella stessa **rete** o è lontana? Questo viene determinato utilizzando la **maschera di rete**. Se gli indirizzi **IP sorgente** e **destinazione**, analizzati tramite la maschera di rete, indicano che sono sulla stessa rete, il **datagramma** può essere inviato direttamente. Altrimenti, il pacchetto deve essere inviato al **router predefinito** per essere instradato verso la destinazione.

Ad esempio, se una macchina ha un indirizzo IP **223.1.1.1** e deve comunicare con **223.1.10.91**, deve prima determinare se questi due indirizzi appartengono alla stessa rete. Se non lo sono, il pacchetto viene inviato al router. Al contrario, se sono nella stessa rete, il pacchetto viene inviato direttamente al destinatario.

Quando si tratta di comunicazione a livello di **collegamento (livello 2)**, le macchine utilizzano **indirizzi MAC**. Se una macchina non conosce l'indirizzo MAC della destinazione, invia un messaggio **broadcast** per

richiedere chi possiede un determinato **indirizzo IP**. La macchina con quell'indirizzo risponde fornendo il proprio **indirizzo MAC**, permettendo la comunicazione diretta.

In contesti con **DHCP**, le macchine non hanno un **indirizzo IP** fisso. Quando una macchina si connette alla rete, invia una richiesta in **broadcast** per ottenere un indirizzo IP da un **server DHCP**. Se il server DHCP è presente nella rete, risponde assegnando un indirizzo IP alla macchina.

Infine, abbiamo introdotto il concetto di **firewall** e il loro ruolo nella **sicurezza della rete**. I firewall operano a diversi livelli del **modello OSI**, proteggendo i dispositivi e le reti da accessi non autorizzati.

25/10

Indirizzi IP in una Rete Locale

Un computer in una rete locale (LAN) può avere più indirizzi IP associati a differenti interfacce di rete o configurazioni. Per esempio, un computer potrebbe avere indirizzi come 192.168.1.2, 192.168.1.3, e così via. Questi indirizzi IP vengono utilizzati per identificare un dispositivo specifico all'interno della rete e permettere la comunicazione tra dispositivi.

Comunicazione tra Dispositivi nella Rete

Quando un dispositivo vuole comunicare con un altro all'interno della stessa rete, utilizza un indirizzo IP locale. Ad esempio, se un computer ha l'indirizzo 192.168.1.4 e vuole comunicare con un altro dispositivo sulla stessa rete, il messaggio viene indirizzato direttamente a quell'indirizzo IP senza necessità di uscire dalla rete locale.

Network Address Translation (NAT)

Il NAT è una tecnica utilizzata per mappare un indirizzo IP pubblico a un indirizzo IP privato. Quando un dispositivo della rete interna invia una richiesta a un server esterno, il router traduce l'indirizzo privato in un indirizzo pubblico. Quando la risposta torna, il router esegue l'operazione inversa, permettendo al dispositivo interno di ricevere la risposta corretta.

Router e MAC Address

I router non solo lavorano con gli indirizzi IP ma anche con gli indirizzi MAC (Media Access Control). Quando un pacchetto arriva al router, il dispositivo utilizza l'indirizzo MAC per identificare il dispositivo di destinazione all'interno della rete locale. Questa operazione avviene a livello 2 del modello OSI.

Switch e Hub

Gli switch e gli hub sono dispositivi utilizzati per connettere più dispositivi all'interno di una rete. Gli switch sono più intelligenti degli hub, poiché possono indirizzare i dati specificamente al dispositivo corretto, riducendo il traffico di rete. Gli hub, invece, inviano i dati a tutti i dispositivi collegati, indipendentemente dalla destinazione.

Firewall e Sicurezza della Rete

I firewall sono dispositivi o software che monitorano e controllano il traffico di rete in entrata e in uscita basandosi su regole di sicurezza predefinite. Essi possono bloccare traffico non autorizzato e proteggere la rete interna da attacchi esterni. Tuttavia, i firewall non proteggono necessariamente da attacchi interni, per questo è importante avere anche altre misure di sicurezza come sistemi di rilevamento delle intrusioni (IDS) e antivirus.

Differenti Livelli di Sicurezza

Le reti possono essere configurate con differenti livelli di sicurezza. Ad esempio, una rete interna può avere un livello di sicurezza più alto rispetto alla rete esterna, impedendo l'accesso non autorizzato dall'esterno ma permettendo comunicazioni interne sicure. I firewall aiutano a gestire queste differenze di sicurezza bloccando il traffico non autorizzato e monitorando le comunicazioni.

29/10

Network Access Control (NAC) e Wireless Network Security

Il **Network Access Control (NAC)** e la **Wireless Network Security** sono due concetti strettamente legati alla sicurezza della rete. NAC si concentra sul controllo dell'accesso alla rete cablata, mentre la wireless network security riguarda la sicurezza delle connessioni via rete wireless. In entrambi i casi, l'obiettivo è proteggere la rete da accessi non autorizzati e garantire che solo dispositivi sicuri possano connettersi.

1. Network Access Control (NAC)

Il Network Access Control è un sistema che gestisce l'accesso alla rete, stabilendo quali dispositivi possano connettersi e quali permessi abbiano all'interno della rete. Oltre a controllare l'autenticazione e l'autorizzazione degli utenti, il NAC esamina anche lo stato di salute dei dispositivi che tentano di connettersi. Questo significa verificare, per esempio, se il sistema operativo è aggiornato, se sono presenti antivirus o se il dispositivo ha altre vulnerabilità.

Componenti Principali del NAC

- **Access Request:** Questo è il metodo con cui un dispositivo tenta di connettersi alla rete. Può trattarsi di un'applicazione, un server, una stampante o qualsiasi dispositivo connesso tramite un IP. I dispositivi possono essere definiti come "Supplicant" o "Client" all'interno del sistema NAC.
- **Backend Systems:** Questi sistemi determinano se il dispositivo può essere autorizzato a connettersi alla rete. Questi sistemi eseguono l'autenticazione e l'autorizzazione, e controllano anche lo stato di salute dei dispositivi.
- **Network Access Server:** Un dispositivo che gestisce i punti di accesso alla rete, come i gateway o i server di autenticazione. Esegue l'autenticazione dei dispositivi remoti e fornisce l'accesso alla rete interna.

Il NAC include vari metodi di controllo come l'autenticazione via 802.1X, l'uso di server DHCP e VLAN per gestire gli indirizzi IP e la segmentazione della rete.

Autenticazione con il Protocollo EAP (Extensible Authentication Protocol)

Il protocollo EAP è una delle tecnologie utilizzate per l'autenticazione all'interno di un sistema NAC. È un framework che supporta diversi metodi di autenticazione, come:

- **TLS** (Transport Layer Security), che usa certificati digitali per autenticare i client e i server.
- **Tunneled TLS:** Una versione di TLS che crea una connessione sicura per l'autenticazione, che può essere utilizzata anche per altre fasi successive dell'autenticazione.

In caso di successo dell'autenticazione, un server di autenticazione (come RADIUS) permette l'accesso al client, gestendo anche la registrazione di eventuali autorizzazioni e log.

2. Wireless Network Security

La **sicurezza delle reti wireless** affronta rischi unici dovuti alla natura delle comunicazioni via onde radio. Le reti wireless sono vulnerabili a vari tipi di attacchi, quindi è fondamentale proteggere l'accesso a questi tipi di reti.

Componenti della Rete Wireless

- **Client (Endpoint):** Dispositivi mobili o altri terminali che si connettono alla rete wireless (smartphone, laptop, tablet).
- **Wireless Access Point (WAP):** Il dispositivo che fornisce l'accesso alla rete tramite segnali radio.
- **Wireless Medium:** Le onde radio che vengono utilizzate per trasmettere i dati tra i dispositivi e l'access point.

Rischi per la Sicurezza nelle Reti Wireless

- **Broadcast Communication:** Le reti wireless utilizzano un canale di comunicazione broadcast, il che significa che i dati sono trasmessi a tutti i dispositivi vicini, aumentando il rischio di intercettazioni.

- **Mobilità:** I dispositivi wireless sono portatili e quindi più vulnerabili a furti o perdite fisiche rispetto a quelli cablati.
- **Risorse Limitate:** I dispositivi wireless spesso hanno capacità di memoria e di elaborazione limitate, il che può renderli vulnerabili ad attacchi come i malware o il DoS (Denial of Service).
- **Accessibilità:** I dispositivi wireless possono trovarsi in luoghi non sicuri e quindi essere vulnerabili a attacchi fisici, oltre che virtuali.

Minacce alle Reti Wireless

Le principali minacce alle reti wireless includono l'intercettazione dei dati, gli attacchi di spoofing (falsificazione di identità), e gli attacchi DoS, tra gli altri. La protezione delle reti wireless richiede l'uso di metodi di crittografia robusti e protocolli di autenticazione sicuri, come WPA3 (Wi-Fi Protected Access 3), che proteggono i dati in transito.

La rete **wireless** si distingue dalla rete **LAN** tradizionale per vari aspetti. In una rete wireless, la connessione avviene tramite un **access point**, che ha un raggio di copertura determinato dalle onde radio. Un problema comune nelle reti wireless è l'**associazione accidentale**, dove un dispositivo si collega involontariamente a una rete vicina invece che a quella desiderata. Un altro rischio è l'**associazione malevola**, in cui un attaccante si finge un access point legittimo per intercettare le password degli utenti e inserirsi nella connessione.

Le **reti peer-to-peer** tra dispositivi wireless che comunicano direttamente senza passare per un access point rappresentano un'altra vulnerabilità. In queste reti, manca un punto centrale di gestione della connessione, aumentando il rischio di minacce alla sicurezza. In contesti più complessi, come le **reti wireless non tradizionali** utilizzate in ambienti universitari o di distribuzione, sono possibili attacchi come i **post-trigger** e **intra-attacchi**, come il furto di identità tramite intercettazione del traffico. In questo scenario, il **social engineering** può essere utilizzato per raccogliere informazioni sui dispositivi o sugli utenti e sfruttarle per compiere attacchi.

Per proteggere le trasmissioni wireless, è essenziale adottare misure contro le minacce come il **dropping dei messaggi**, la **modifica del contenuto dei messaggi** e l'**interruzione del servizio**. Le principali contromisure includono l'uso di **signal jammer** per impedire l'intercettazione del segnale e l'adozione di tecniche di **crittografia dei dati**, come la protezione delle chiavi utilizzate per cifrare le comunicazioni.

Il protocollo **802.1X** è uno degli strumenti per evitare l'accesso non autorizzato alla rete, autenticando i dispositivi tramite porte logiche. Per una protezione aggiuntiva, è consigliabile l'uso della **crittografia**, l'adozione di **antivirus**, **antispider**, **firewall** e il cambio delle password predefinite e degli identificatori di rete.

Un altro aspetto da considerare riguarda la sicurezza dei **dispositivi mobili**, sempre più utilizzati nelle infrastrutture aziendali. I dispositivi mobili rubati o quelli a cui terze persone riescono ad accedere rappresentano una minaccia significativa, poiché spesso mancano dei necessari controlli di sicurezza. Inoltre, molte applicazioni mobili non sono sicure e possono essere vettori per **malware**, specialmente quando interagiscono con sistemi esterni come moduli di sincronizzazione o applicazioni di terze parti.

L'approccio alla sicurezza sui dispositivi mobili deve includere misure come **autenticazione forte**, l'uso di **VPN**, **firewall** e la protezione dei dati in transito. Le politiche di sicurezza devono essere aggiornate per far fronte alla crescente mobilità e interconnessione tra dispositivi aziendali e personali, senza dare per scontato che ogni dispositivo mobile sia sicuro di default.

Per quanto riguarda le **reti wireless**, esse presentano numerosi vantaggi, come la riduzione dei costi e la mobilità delle stazioni, ma anche sfide legate al fatto che i dati vengono trasmessi tramite onde radio (da **2.4 GHz** a **5 GHz**), che possono essere vulnerabili ad attacchi come il **cross-site scripting** o l'iniezione di pacchetti di dati. Inoltre, essendo reti **broadcast**, i dati trasmessi possono essere intercettati da più dispositivi situati nella stessa area di copertura, aumentando i rischi di sicurezza.

Le **tecnologie wireless**, come quelle basate sugli standard **IEEE 802.11**, si sono evolute nel tempo. Dal **802.11b** (con una velocità massima di 2 Mbps), si è passati al **802.11a** (che raggiunge i 54 Mbps), e successivamente al **802.11g**, che supporta anch'esso fino a 54 Mbps. Ogni aggiornamento ha cercato di

migliorare le prestazioni e la sicurezza, affrontando le problematiche legate alle interferenze e alle limitazioni di banda.

Il **MAC protocol** (Medium Access Control) è responsabile della gestione dell'accesso alla rete LAN e dei messaggi trasmessi tra i vari dispositivi. Esso si compone di più unità: il **MAC control**, che contiene informazioni necessarie al protocollo, come quelle relative alla sorgente e alla destinazione del messaggio, e i **dati**, che provengono dal layer superiore.

L'ultima parte del frame è il **Frame Check Sequence (FCS)**, un campo che contiene il **CRC** (Cyclic Redundancy Check), utilizzato per la rilevazione degli errori. Questo codice viene calcolato dal mittente sui bit del messaggio e aggiunto al frame. Il ricevente calcola il CRC del messaggio e confronta il valore ottenuto con quello trasmesso. Se i due valori coincidono, il messaggio è integro, altrimenti uno o più bit sono stati alterati.

Il **Logical Link Control (LLC)** è l'ultimo layer del protocollo, ed è responsabile della gestione degli errori e del flusso dei dati. Esso tiene traccia dei messaggi correttamente inviati e trasmette quelli che presentano errori.

Un esempio di **MAC protocol data unit (PDU)** per 802.3 include campi come l'indirizzo MAC del mittente, quello del destinatario e i dati trasmessi. Per la rete wireless, invece, si utilizzano frame 802.11, che presentano campi aggiuntivi per indirizzare i vari componenti del sistema, come gli access point e i dispositivi ricevitori.

Per quanto riguarda la **rete wireless**, essa è costituita da diversi componenti:

- **Basic Service Set (BSS)**: un insieme di stazioni wireless che usano lo stesso protocollo MAC e competono per l'accesso alla rete.
- **Distribution System (DS)**: il sistema che collega vari BSS per formare una rete più ampia.
- **Access Points (AP)**: dispositivi che permettono alle stazioni di accedere alla rete wireless, fungendo da bridge tra la stazione e il DS.

Per la gestione della rete e della sicurezza, si utilizzano diversi servizi, come l'integrazione e la distribuzione, che garantiscono il corretto trasferimento dei dati tra stazioni e il **Distribution System**. L'**associazione** è il processo che permette a una stazione di connettersi a un access point. Esistono anche diverse modalità di **transizione**: la **non-transmission** (quando la stazione è stazionaria), la **BSS transition** (quando una stazione si sposta all'interno dello stesso BSS) e la **BSS transition tra SS diversi**.

La sicurezza nella rete wireless è garantita dallo standard **WPA** (Wi-Fi Protected Access), che ha sostituito il vulnerabile **WEP**. La versione avanzata **WPA2** include tecniche come la gestione dinamica delle chiavi crittografiche, migliorando la sicurezza della comunicazione.

Infine, il processo di **autenticazione e gestione delle chiavi** è essenziale per stabilire una connessione sicura. Le fasi principali del processo sono:

- **Discovery**: l'access point invia messaggi per identificarsi come supportante uno specifico standard di sicurezza.
- **Authentication**: la stazione e l'authentication server si scambiano le credenziali per autenticarsi reciprocamente.
- **Key Management**: generazione e gestione delle chiavi crittografiche per proteggere i dati.
- **Protected Data Transfer**: trasferimento sicuro dei dati tra stazione e access point.

Il protocollo di **terminazione della connessione** riporta la connessione allo stato originale, garantendo la sicurezza nella disconnessione.

30/10

Durante la fase di **discovery** di una rete wireless, la stazione che desidera connettersi invia diversi messaggi all'**access point** e all'**authentication server**. Questi messaggi sono fondamentali per stabilire

una connessione sicura. L'**authentication server** può trovarsi direttamente nell'**access point** o nel **Distribution System**.

Il primo messaggio che la stazione invia è una **probe request**, in cui comunica all'access point il desiderio di connettersi alla rete. L'access point risponde con una **probe response**, che include i parametri di sicurezza che l'access point supporta, come le sue **capabilities** di sicurezza.

I messaggi successivi, come l'**Open System Authentication Request** e la **Response**, sono usati per scambiarsi gli identificatori e non per eseguire una vera e propria autenticazione. In queste fasi, la stazione e l'access point si scambiano messaggi di verifica, ma non avviene ancora una verifica dei permessi di accesso.

L'ultima fase del processo di discovery è l'**associazione**, che permette alla stazione di collegarsi formalmente all'access point. La stazione invia una richiesta di associazione, includendo i parametri di sicurezza che ha scelto in base a quelli ricevuti dall'access point. L'access point risponde confermando la sicurezza associata e completando il processo.

Dopo la fase di discovery, si passa alla **fase di autenticazione**, che utilizza il protocollo 802.1X. In questa fase, vengono utilizzate **porte controllate** e **porte non controllate**: nelle porte non controllate passa il traffico delle stazioni non ancora autenticate, mentre nelle porte controllate passa il traffico solo delle stazioni autorizzate.

Durante l'autenticazione, la stazione invia un messaggio **E-APP** all'authentication server, il quale risponde in base al protocollo di autenticazione scelto (ad esempio TLS o DTLS). A questo punto, l'authentication server invia all'access point informazioni sulla chiave generata, che certificano l'autenticazione della stazione. L'access point a sua volta invia queste informazioni alla stazione.

Fino a questo momento, le porte controllate sono ancora bloccate, poiché la chiave di autenticazione non è ancora stata generata o salvata da entrambe le parti. Una volta completata l'autenticazione, vengono generate due principali tipi di chiavi: **pair-wise keys** per la comunicazione tra stazione e access point, e **group keys** per la comunicazione multicast.

La **pair-wise key** viene generata tramite una **master session key** o una **pre-shared key** (PSK). Se la PSK viene utilizzata, la pair-wise master key (PMK) è identica alla PSK. Se invece si usa la master session key, questa può essere troncata per ottenere la PMK, che serve per generare ulteriori chiavi **pair-wise transient key** (PTK).

Una volta che la PMK è generata, vengono prodotte tre chiavi: la **Key Confirmation Key (KCK)** per garantire l'integrità dei messaggi, la **Key Encryption Key (KEK)** per proteggere le chiavi e i dati durante la trasmissione, e la **Temporal Key (TK)** per proteggere il traffico tra la stazione e l'access point.

Le **group keys**, invece, vengono utilizzate per la comunicazione multicast e non sono generate durante l'autenticazione tra la stazione e l'access point, ma direttamente dall'access point. Queste chiavi sono trasmesse a tutte le stazioni e vengono utilizzate per la sicurezza dei messaggi inviati a più dispositivi contemporaneamente.

Quando una stazione si disconnette, la **group key** e la **group master key** vengono modificate per garantire la sicurezza della rete. Le **pair-wise keys**, invece, sono specifiche per ogni coppia di dispositivi, quindi ogni stazione avrà la sua chiave univoca con l'access point.

In seguito alla generazione delle chiavi, il traffico tra la stazione e l'access point può passare attraverso le **porte controllate**. Il processo di **4-Way Handshake** assicura che le chiavi siano correttamente generate e condivise tra le due parti. Durante questo processo, vengono scambiati **nonce** e messaggi per confermare che le chiavi sono sicure e che non ci sono attacchi in corso, come gli attacchi **man-in-the-middle**.

Quando le chiavi sono stabilite, la stazione e l'access point sono in grado di generare le **Group Temporal Key (GTK)** per la comunicazione multicast. La GTK è cifrata e trasmessa in modo sicuro all'interno della rete, e ogni stazione deve decifrarla per poter partecipare alla comunicazione.

Il **Protected Data Transfer Phase** è l'ultima fase del processo, durante la quale i messaggi vengono inviati in modo protetto. Ci sono due metodi di protezione: il **Temporal Key Integrity Protocol (TKIP)** e il

Counter Mode with Cipher Block Chaining Message Authentication Code Protocol (CCMP). Entrambi forniscono l'integrità e la confidenzialità dei dati, ma CCMP è considerato più sicuro, poiché utilizza **AES** per cifrare i messaggi.

Infine, la funzione **pseudorandom** è utilizzata per generare i **nonce** e le chiavi, e garantisce che la sicurezza dei dati venga mantenuta nel tempo, evitando la predicibilità dei valori generati.

WIRESHARK

Wireshark è uno strumento di analisi del traffico di rete che cattura i pacchetti di dati trasmessi su una rete e consente di esaminarli in dettaglio. Originariamente sviluppato nel 1997 da Gerald Combs con il nome di **Ethereal**, è stato rinominato **Wireshark** nel 2006 a causa di problemi con il marchio "Ethereal". È uno **sniffer di pacchetti**, il che significa che cattura passivamente i dati che transitano attraverso una rete senza alterare il traffico stesso.

Funzionalità principali:

- **Packet Capture Library:** Riceve copie di ogni pacchetto a diversi livelli del protocollo TCP/IP, sia in ingresso che in uscita, da un determinato computer, utilizzando interfacce come Ethernet o Wi-Fi.
- **Packet Analyzer:** Analizza i pacchetti catturati e visualizza il loro contenuto, permettendo di esaminare i dati nel contesto di vari protocolli di rete.

Wireshark permette di:

- Monitorare e analizzare il traffico di rete in tempo reale o tramite file di cattura precedentemente salvati.
- Supporta vari formati di output (ad esempio XML, CSV).
- Disporre di un'interfaccia grafica user-friendly.

Alcuni dei principali filtri in Wireshark:

1. **Filtrare pacchetti in base a un determinato IP** (sia sorgente che destinazione).
2. **Filtrare le conversazioni tra due IP.**
3. **Filtrare pacchetti TCP che utilizzano una specifica porta.**
4. **Escludere determinati protocolli (es. HTTP, DNS, etc.).**
5. **Filtrare pacchetti TCP che contengono una determinata parola.**

Analisi di HTTP con Wireshark:

Durante un esercizio pratico, si possono analizzare pacchetti relativi al protocollo HTTP, utilizzando il filtro HTTP per estrarre solo i pacchetti pertinenti. Ad esempio, si può verificare:

- La versione di HTTP utilizzata (ad esempio, HTTP/1.1 o HTTP/2).
- L'indirizzo IP del client (browser) e del server.
- Il codice di stato HTTP (ad esempio, 200 OK, 304 Not Modified).
- La dimensione del contenuto restituito dal server.

Wireshark permette di esplorare anche l'header HTTP e la gestione delle richieste condizionali, come nel caso del campo If-Modified-Since, che consente al server di inviare la risorsa solo se è stata modificata dopo una determinata data.

Esercitazioni pratiche:

Le esercitazioni in Wireshark spesso implicano l'analisi di traffico HTTP. Gli studenti devono identificare vari dettagli come la versione HTTP, i campi dell'header, le risposte del server, e la gestione delle risorse con il codice di stato.

In una tipica esercitazione, si cattura il traffico da una sessione web e si analizzano i vari pacchetti, rispondendo a domande su:

- Versioni di HTTP.
- IP sorgente e destinatario.
- Risposte del server e codice di stato.
- Dati modificati o non modificati.

Wireshark è uno strumento potente per la diagnosi e l'analisi del traffico di rete, utile in vari contesti, dalla risoluzione dei problemi di rete alla rilevazione di potenziali attacchi informatici.

20/11

I firewall sono dispositivi fondamentali nella gestione della sicurezza delle reti. Questi possono operare a diversi livelli del modello OSI, con regole di filtraggio che vengono applicate per determinare se un pacchetto di dati può attraversare la rete. Le regole tipiche si basano su quattro componenti principali: l'indirizzo IP sorgente, la porta sorgente, l'indirizzo IP di destinazione e la porta di destinazione.

Le regole di un firewall possono essere semplici, come nel caso di un controllo di pacchetti a livello 3-4 del modello OSI, ma presentano dei limiti. Queste regole sono utili per gestire traffico IP, che è solitamente il tipo di traffico principale. Tuttavia, a questo livello, non è possibile effettuare operazioni più avanzate come l'autenticazione dell'utente o il filtraggio a livello applicativo.

Un altro punto critico riguarda il logging. I firewall tradizionali loggano solo informazioni relative ai pacchetti a livello 3 o 4, mentre per un controllo più preciso e dettagliato è necessario ricorrere a soluzioni più avanzate. L'azione di un firewall può essere "vietato" o "scartato". Se un pacchetto è vietato, viene inviata una risposta informando che il pacchetto non è stato accettato; se il pacchetto è scartato, non viene inviata alcuna risposta, rendendo il processo più "silenzioso".

Un esempio di una regola tipica di firewall può essere una "regola di default", che vieta tutto il traffico se non specificato diversamente, applicando l'azione in base alla corrispondenza dei pacchetti con i parametri delle regole.

Alcuni firewall sono configurati in modo tale che permettono il traffico interno (da una rete privata) verso qualsiasi porta, ma la gestione più rigida può prevedere il controllo di accessi più restrittivi per migliorare la sicurezza. In situazioni più avanzate, si configurano anche regole specifiche per la protezione di macchine sensibili, come nel caso dei firewall che impediscono l'accesso alla macchina stessa per evitare che venga compromessa.

A livello applicativo, le regole sono più complesse. Ad esempio, si possono configurare server proxy che agiscono come firewall di livello 7, il che significa che possono analizzare e filtrare il traffico a livello delle applicazioni (come HTTP o email). Questi firewall più avanzati possono anche gestire l'autenticazione dell'utente e applicare politiche di accesso specifiche, come l'accesso a determinate risorse solo in determinati orari o in risposta a comandi specifici.

Tuttavia, un firewall di livello applicativo comporta una maggiore complessità e un carico maggiore, poiché le regole devono essere adattate a ciascun tipo di protocollo o applicazione. Per esempio, se si desidera gestire il traffico HTTP, si dovrà configurare un proxy HTTP specifico.

Un altro aspetto importante riguarda il comportamento dei firewall stateful, che creano automaticamente voci nella tabella delle connessioni ogni volta che un pacchetto esce. Quando la risposta ritorna, viene riconosciuta e permessa, e la regola viene rimossa dopo il completamento della connessione.

Introduzione alla Configurazione di Reti e Firewall

Abbiamo diverse configurazioni di rete, alcune delle quali includono **due firewall** che separano le reti interne ed esterne. In un altro scenario, invece, si può osservare che le reti interne non passano attraverso entrambi i firewall per accedere all'esterno, ma solo attraverso uno di essi. Questo tipo di scelta viene

effettuato per ottimizzare i **costi** e ridurre la necessità di **macchine separate**. Un esempio di tale scelta è l'acquisto di un **server più potente** con **tre interfacce di rete**, che riduce i costi rispetto a soluzioni multiple.

Il Ruolo della VPN

La **VPN (Virtual Private Network)** è una tecnologia che consente di creare un canale cifrato, assicurando che il traffico tra il **PC di casa** e i **server aziendali** non possa essere intercettato. L'uso della VPN consente di accedere alle risorse aziendali, come le **stampanti** aziendali, anche da remoto. Tra i protocolli principali per la VPN, ricordiamo **PPTP (Point-to-Point Tunneling Protocol)** e **IPsec**. Questi protocolli offrono **funzionalità crittografiche** per proteggere il traffico, sebbene con differenze nella **codifica** dei dati.

IDS (Intrusion Detection System) e il Traffico di Rete

Un **IDS (Intrusion Detection System)** è fondamentale per il monitoraggio dei pacchetti di rete. Se un pacchetto sospetto viene rilevato, il sistema avvisa e può **bloccare** il traffico indesiderato. La posizione dell'IDS è una scelta importante, poiché può essere installato in vari punti della rete: sulla rete interna, su quella esterna o su una **rete DMZ**. La decisione dipende da vari fattori, tra cui la **sicurezza** e l'**efficienza dei costi**.

La Scelta della Posizione per l'IDS

Quando si decide dove posizionare un **IDS**, è importante considerare il traffico che deve essere monitorato. Ad esempio, se l'IDS è posizionato vicino alla **rete interna**, avrà accesso a tutto il traffico proveniente dall'interno e verso l'esterno. Tuttavia, se si posiziona vicino alla **DMZ**, si avrà il controllo del traffico che entra e esce da un'area perimetrale. La posizione migliore dipende dalla **criticità** delle risorse da proteggere.

DNS: Funzioni e Posizionamento

Il **DNS (Domain Name System)** è fondamentale per la risoluzione dei nomi di dominio, consentendo ai dispositivi di tradurre i **nomi di dominio** in indirizzi IP. Esistono due tipi di DNS: uno **autoritativo** per l'esterno, che risolve i domini per l'accesso esterno, e uno per l'interno, che traduce i nomi di dominio per la **rete locale**. È essenziale garantire che il DNS pubblico sia accessibile dall'esterno, mentre quello interno risolve solo i nomi per le macchine interne.

Configurazione dei Server e delle Applicazioni

La configurazione di un server per un'applicazione, come un **software di contabilità**, dipende da chi deve accedere a quella risorsa. Se l'applicazione è destinata solo agli **utenti interni**, il server può essere collocato sulla rete **interna**. Se invece deve essere accessibile anche dall'esterno, può essere posizionato nella **DMZ**.

Considerazioni sul Firewall e le Regole

Le **regole del firewall** determinano quali traffico è consentito o bloccato. Le regole possono variare in base alla posizione del firewall e alla **rete** che proteggono. Se la regola riguarda l'accesso da una rete interna a una risorsa, la regola va applicata sul firewall che separa quella rete dalla risorsa. Ogni firewall ha regole specifiche, ma le **regole generali** rimangono consistenti.

Conclusioni sulle Reti Aziendali

La configurazione delle **reti aziendali** e l'implementazione di **firewall** e **IDS** sono processi cruciali per garantire la sicurezza delle comunicazioni interne ed esterne. La scelta delle **posizioni** dei server, del DNS e dell'IDS deve essere fatta tenendo conto delle necessità aziendali, del traffico da monitorare e dei costi da sostenere. Una corretta **pianificazione delle regole del firewall** è fondamentale per il controllo sicuro e il corretto funzionamento della rete aziendale.

Posizionamento e ordine delle regole nel firewall

Una delle prime considerazioni nella configurazione di un firewall riguarda il posizionamento delle regole di sicurezza. Ad esempio, quando si decide di bloccare l'accesso a una risorsa specifica, come nel caso di

una risorsa denominata "elettrospost", è importante applicare la regola il più presto possibile nel processo di filtraggio del traffico. Questo consente di bloccare immediatamente i pacchetti indesiderati, evitando che essi attraversino livelli successivi senza necessità di ulteriori controlli. In altre parole, mettere una regola di blocco all'inizio del processo di controllo del traffico migliora l'efficienza del sistema e riduce i tempi di elaborazione.

Autenticazione degli accessi

Un altro aspetto cruciale nella gestione della sicurezza di una rete riguarda l'autenticazione degli accessi. Se un firewall consente l'accesso alla rete, è essenziale che tale accesso sia autenticato. Ciò significa che solo gli utenti o dispositivi autorizzati possono attraversare il firewall e accedere alle risorse protette. In pratica, per consentire l'accesso, potrebbe essere necessario utilizzare un client dedicato che permetta l'interazione con il firewall, come ad esempio un software di gestione per l'accesso remoto tramite Wi-Fi o DSL.

Gestione del traffico e blocco dei pacchetti indesiderati

Una volta stabilito un sistema di autenticazione, è fondamentale gestire con attenzione il tipo di traffico che il firewall deve consentire o bloccare. Alcuni pacchetti, come quelli provenienti da interfacce non autorizzate, possono essere utilizzati da attaccanti per compromettere la sicurezza della rete. Pertanto, è importante configurare il firewall in modo da bloccare traffico sospetto, come quello di tipo "broadcast", che potrebbe essere utilizzato in attacchi di tipo "Source Looping". In particolare, il traffico ICMP (Internet Control Message Protocol), utilizzato per il ping e altri messaggi di controllo, deve essere trattato con cautela, poiché potrebbe essere sfruttato per attività dannose.

Protocollo SMP e traffico di rete

Il traffico relativo al protocollo SMP (Sync on Network Management Protocol) deve essere gestito con attenzione. Questo protocollo è utilizzato per la gestione della rete e per la sincronizzazione di dispositivi. Tuttavia, dovrebbe essere consentito solo a client autenticati, per evitare che dispositivi non autorizzati possano inviare comandi o raccogliere informazioni sensibili sulla rete. È quindi necessario configurare il firewall affinché il traffico SMP venga filtrato in modo appropriato, impedendo accessi non autorizzati.

Gestione dei log

Un altro aspetto fondamentale nella gestione della sicurezza della rete riguarda la registrazione dei log. I log sono utili per diagnosticare problemi di sicurezza, monitorare attività sospette e analizzare eventuali attacchi. È importante configurare il firewall in modo che registri correttamente tutte le azioni eseguite e che i file di log vengano mantenuti per un periodo di tempo adeguato. Nonostante i log possano crescere in dimensioni, l'archiviazione di questi file non dovrebbe rappresentare un problema, poiché i costi di storage sono relativamente bassi. La rotazione dei log è una pratica importante che permette di mantenere i dati rilevanti e gestire al meglio lo spazio di archiviazione.

Firewall hardware vs. firewall software

Infine, è importante considerare le diverse opzioni di firewall disponibili: hardware e software. Un firewall hardware è preconfigurato e generalmente offre una protezione più robusta, mentre un firewall software può essere installato su una macchina generica, ma richiede configurazioni più dettagliate. La manutenzione del firewall, che prevede l'aggiornamento delle regole e la gestione degli accessi, può essere eseguita tramite interfacce grafiche o a linea di comando. La configurazione tramite linea di comando è spesso preferita per la sua precisione e flessibilità, ma richiede una maggiore conoscenza tecnica.

Conclusioni

In sintesi, un buon sistema di firewall deve essere configurato in modo da garantire la massima sicurezza della rete. Le regole di sicurezza devono essere posizionate correttamente per ottimizzare le performance del firewall, l'autenticazione degli accessi è fondamentale per proteggere le risorse della rete, e la gestione del traffico deve prevedere il blocco di pacchetti indesiderati. Inoltre, la registrazione dei log e la scelta tra firewall hardware e software sono aspetti che contribuiscono a mantenere un ambiente di rete sicuro ed efficiente.

Sicurezza nelle Reti Aziendali: Firewall, IDS e IPS

Nelle piccole aziende, solitamente si trova un solo firewall che funge da punto di ingresso per la rete, il quale può gestire una rete piatta. Tuttavia, quando si lavora con reti aziendali più complesse, è necessario implementare politiche di sicurezza più rigorose. Una delle raccomandazioni fondamentali è che le misure di sicurezza devono essere implementate immediatamente, non appena si individuano comportamenti vietati. Per esempio, se c'è qualcosa che deve essere vietato, è importante applicare il blocco il prima possibile, sia che si tratti di accessi in uscita che in entrata. Ogni configurazione deve essere chiusa e sicura, specialmente in caso di attacchi o configurazioni errate del router. È sempre utile avere un backup, come un altro computer, pronto a intervenire in caso di problemi.

Le regole di sicurezza a livello di firewall si dividono in due principali categorie:

1. **Livello di rete (Layer 3):** Queste regole sono progettate per gestire il traffico in base agli indirizzi IP e alle porte di comunicazione.
2. **Livello applicativo:** Qui le regole gestiscono il traffico basato sulle applicazioni in esecuzione nella rete, come i DNS.

Un aspetto cruciale nella gestione delle reti aziendali è la distinzione tra **DNS esterni e interni**. I DNS esterni sono utilizzati per tradurre i nomi dominio per l'intero mondo, mentre i DNS interni sono riservati per la traduzione dei nomi all'interno della rete aziendale. Inoltre, i router spesso utilizzano il NAT (Network Address Translation) per gestire il traffico.

L'amministratore di rete può accedere al router attraverso un client specifico, configurando il firewall per consentire l'accesso solo su porte specifiche e utilizzando un sistema di logging per tracciare tutte le attività. Un'altra buona prassi è l'utilizzo di un server **syslog** per centralizzare i log delle attività di rete.

Il firewall, pur essendo una componente essenziale, non deve essere considerato la prima linea di difesa. Ogni macchina nella rete deve essere configurata come se il firewall non esistesse, garantendo che tutte le protezioni interne siano adeguate.

Protezione delle Macchine e Utilizzo di Proxy

Ogni utente dovrebbe avere un **firewall personale** per proteggere il proprio dispositivo. È importante verificare periodicamente che il firewall sia configurato correttamente, poiché durante l'installazione e la disinstallazione di applicazioni possono essere aggiunte regole che, tuttavia, non vengono sempre rimosse. In questo caso, è fondamentale monitorare e correggere queste configurazioni.

L'uso di **proxy applicativi** può essere utile per migliorare la sicurezza, in quanto permette di monitorare e controllare l'accesso alle applicazioni, garantendo che solo gli utenti autorizzati possano interagire con esse.

IDS e IPS: Sistemi di Rilevamento e Prevenzione delle Intrusioni

Gli **IDS (Intrusion Detection Systems)** e gli **IPS (Intrusion Prevention Systems)** sono strumenti complementari ai firewall, utilizzati per monitorare e proteggere la rete aziendale.

1. **IDS:** Questi sistemi automatizzano il controllo dei pacchetti di rete per rilevare attività sospette o dannose. Quando un incidente viene identificato, l'IDS invia una notifica al personale di gestione. Sebbene l'IDS non blocchi attivamente il traffico, registra gli eventi e produce report che possono essere utilizzati per investigare e documentare gli incidenti. Gli incidenti devono essere sempre riportati e analizzati per migliorare la sicurezza.
2. **IPS:** A differenza dell'IDS, l'IPS ha la capacità di bloccare attivamente il traffico dannoso. Può essere configurato per terminare sessioni sospette o bloccare il traffico in base a regole predefinite. Gli IPS sono più efficaci quando vengono configurati per lavorare in sinergia con altre soluzioni di sicurezza, come il firewall, in modo da prevenire e mitigare gli attacchi in tempo reale.

Tipologie di IDS e IPS

Esistono diverse tipologie di **IDS e IPS**:

- **IDS basati su firma**: Rilevano attacchi noti attraverso la corrispondenza con firme di attacchi precedenti.
- **IDS basati su anomalie**: Creano una "baseline" del traffico di rete e identificano eventuali deviazioni da questa baseline, segnalando attività sospette.
- **IDS specifici per protocollo**: Monitorano il traffico relativo a specifici protocolli (ad esempio HTTP, FTP) e segnalano anomalie nel loro funzionamento.

Al fine di migliorare la protezione, è spesso consigliato utilizzare una combinazione di diverse tipologie di IDS e IPS, se il budget lo consente, per ottenere una sicurezza più robusta.

Signature-Based Detection è una tecnica molto utile per rilevare attacchi già conosciuti. Funziona bene quando un attacco è già stato identificato, perché riconosce schemi e firme già noti. Il processo è molto semplice, ma la sua limitazione è che non può rilevare attacchi nuovi o sconosciuti. Per esempio, se un attaccante modifica la firma dell'attacco, questa tecnologia potrebbe non riconoscerlo, ed è per questo che il "Zero-Day" (attacchi mai visti prima) è un grande problema per questo tipo di rilevamento.

Al contrario, **Anomaly-Based Detection** si concentra sul traffico di rete "anomalous", cercando deviazioni rispetto al comportamento normale. Per esempio, se normalmente invii 10 email al giorno, ma un giorno ne invii 1.000, il sistema può interpretarlo come un possibile attacco, come un virus. Un grande vantaggio di questa tecnica è che può rilevare attacchi nuovi, ma richiede un allenamento adeguato del sistema, altrimenti rischia di generare troppi falsi positivi o negativi.

Un aspetto interessante del sistema è che può essere configurato per funzionare in modalità dinamica, cioè adattandosi continuamente al cambiamento delle condizioni aziendali. Tuttavia, bisogna fare attenzione, poiché un allenamento errato o troppo specifico potrebbe comportare un rischio, come ad esempio un attacco che, se rilevato in un ambiente diverso da quello addestrato, potrebbe passare inosservato.

L'**IDS** può essere utilizzato insieme a un server di **gestione** (management server) che raccoglie e analizza i dati. In una rete aziendale più complessa, i **sensori** possono essere distribuiti e utilizzano un **database** per memorizzare i log degli eventi rilevati, così da permettere una ricerca efficiente.

L'**alerting** è una funzione chiave di questi sistemi. Quando viene rilevato un evento sospetto, un **alert** viene generato e può essere inviato a un amministratore di sistema per un'ulteriore analisi. La gestione delle **blacklist** e **whitelist** aiuta a distinguere tra comportamenti noti e sicuri, e comportamenti potenzialmente dannosi. Una buona configurazione dei log e degli alert è essenziale per evitare che gli amministratori ignorino eventuali problemi a causa di un numero eccessivo di notifiche.

Quando si parla di **IPS** (Intrusion Prevention Systems), la differenza principale con gli IDS è che questi sistemi non solo rilevano, ma possono anche **prevenire** gli attacchi, bloccando direttamente il traffico maligno. Questo è possibile grazie alla configurazione "in-line", dove il traffico passa attraverso il sistema di rilevamento prima di raggiungere la rete interna.

Un altro concetto importante è la gestione del traffico **wireless**. Poiché molte reti moderne utilizzano tecnologie **Wi-Fi**, è essenziale che gli IDS e IPS possano monitorare anche il traffico **wireless**. Questi sistemi possono essere configurati per raccogliere il traffico da specifici **access point** o **switch**, ma è fondamentale che i sensori possano ascoltare tutto il traffico per identificare potenziali minacce. Inoltre, la **sicurezza wireless** è un tema importante, e tecnologie come **WEP**, **WPA**, e **WPA2** sono utilizzate per proteggere le connessioni.

Infine, la sincronizzazione dei **log** attraverso l'uso di **NTP (Network Time Protocol)** è fondamentale per assicurarsi che tutte le macchine siano allineate con lo stesso orario, al fine di correlare correttamente gli eventi di rete. In un contesto aziendale, la gestione centralizzata delle informazioni raccolte attraverso sensori e log è fondamentale per una risposta tempestiva agli attacchi.

Il sistema di rilevamento delle intrusioni (IDS, Intrusion Detection System) può essere suddiviso in due grandi categorie: basati su regole e basati su rilevamento delle anomalie. Gli IDS basati su regole confrontano il traffico con un insieme di regole prestabilite e generano un allarme quando rilevano un comportamento che corrisponde a un attacco noto. D'altra parte, gli IDS basati su anomalie si concentrano su comportamenti che si discostano dalla norma, identificando attività inusuali come potenziali minacce.

Un tipo di IDS molto comune è l'**host-based IDS (HIDS)**, che monitora non il traffico di rete, ma le attività interne di una singola macchina. Questo sistema è particolarmente utile per rilevare attacchi che non sono facilmente visibili attraverso il monitoraggio del traffico di rete, come modifiche sospette ai file di sistema o attività anomale nei processi locali. Gli HIDS sono spesso utilizzati su macchine molto critiche per l'azienda, come i server di posta elettronica o i server per applicazioni aziendali vitali.

Le prestazioni di una macchina possono risentire dell'implementazione di un HIDS, poiché il sistema monitora costantemente le attività della macchina e le chiamate di sistema. Di solito, per gestire questa risorsa in modo efficace, in ambienti aziendali più grandi viene utilizzato un **management server** che centralizza la gestione di più IDS distribuiti su diverse macchine.

Caratteristiche degli Host-Based IDS:

1. **Monitoraggio dei processi locali:** Gli HIDS osservano tutti gli eventi locali della macchina, come chiamate ai sistemi e modifiche ai file.
2. **Funzionalità di logging avanzato:** Grazie al monitoraggio dettagliato delle attività, gli HIDS offrono una registrazione completa (logging) delle operazioni della macchina, rendendo più facile rilevare attività sospette.
3. **Impatto sulle performance:** Poiché gli HIDS monitorano costantemente la macchina, le risorse vengono utilizzate in modo intensivo, rallentando il sistema se la macchina non è sufficientemente potente.
4. **Sicurezza avanzata:** Oltre al traffico di rete, gli HIDS possono monitorare e rilevare minacce come buffer overflow, modifiche ai file e attività di sistema anomale.

Autenticazione e Sicurezza

L'autenticazione è un passo cruciale nella sicurezza informatica. Prima che un utente possa accedere a un sistema, è necessario verificarne l'identità attraverso processi come l'autenticazione tramite **username e password**, **biometria**, **token fisici** o altri metodi.

Fasi fondamentali della sicurezza informatica:

1. **Identificazione:** Riconoscere chi è l'utente o la macchina.
2. **Autenticazione:** Verificare che l'utente sia chi dice di essere.
3. **Autorizzazione:** Permettere l'accesso solo alle risorse per cui l'utente è autorizzato.

L'autenticazione può avvenire tramite vari metodi, tra cui:

- **Autenticazione tramite password:** Il metodo più tradizionale, ma vulnerabile se non protetto adeguatamente. Le password devono essere complesse e memorizzate in modo sicuro (ad esempio, tramite hash con sale per evitare conflitti tra password uguali).
- **Autenticazione basata su possesso:** Un esempio è l'uso di un token fisico, come una smart card o un dispositivo che genera una password temporanea per l'accesso.
- **Autenticazione biometrica:** Include metodi come l'identificazione facciale o la scansione dell'impronta digitale.

In azienda, il sistema di gestione delle credenziali deve garantire che solo gli utenti legittimi abbiano accesso alle risorse sensibili, e un accesso sicuro viene solitamente implementato utilizzando tecniche di **verifica a due fattori (2FA)**.

Sicurezza delle password

La sicurezza delle password è essenziale per evitare attacchi come il furto di credenziali o l'accesso non autorizzato. Una password debole è una delle vulnerabilità più comuni. Per garantire la sicurezza, le password dovrebbero essere:

- **Complesse:** Una combinazione di lettere, numeri e simboli.
- **Lunghe:** Più lunga è una password, più difficile è da indovinare.
- **Uniche:** Evitare l'uso di parole comuni o sequenze facilmente indovinabili.

Le password devono essere trattate in modo sicuro, memorizzandole in formato hash (con l'aggiunta di un sale per rendere ogni hash unico) e mai in chiaro. Inoltre, è fondamentale proteggere i file contenenti le password per evitare accessi non autorizzati.

Il **riconoscimento biometrico** è una tecnologia interessante e utile, ma presenta alcune problematiche legate alla **precisione** dei sistemi digitali che memorizzano e confrontano le **caratteristiche fisico-comportamentali** degli individui. Questi sistemi si basano sull'acquisizione di **dati biometrici**, come **impronte digitali**, che vengono poi confrontati con le informazioni precedentemente memorizzate. Tuttavia, le caratteristiche biometriche possono variare nel tempo, anche se in misura minima, il che rende difficile garantire una corrispondenza perfetta ogni volta che vengono acquisite.

Variazioni nei Dati Biometrici

Per esempio, quando si acquisisce un'impronta digitale, la **misura della lunghezza** del dito o la **forma della falange** può variare leggermente rispetto ai dati memorizzati. Questi piccoli cambiamenti, anche se impercettibili, possono influenzare il confronto tra il pattern acquisito e quello salvato. Una leggera differenza può alterare i valori biometrici e creare una distanza che indica una mancata corrispondenza. Questo porta alla necessità di definire una **"soglia di accettazione"**, che determina la **tolleranza** alle differenze nei dati biometrici. La soglia stabilisce quanto può essere piccola o grande la differenza tra i dati biometrici acquisiti e quelli memorizzati prima di considerare il confronto come non valido.

La Soglia di Accettazione

La **soglia di accettazione** è fondamentale per il funzionamento di un sistema biometrico. Se la soglia è troppo alta, potrebbero verificarsi troppi **falsi positivi**, ossia persone non autorizzate che riescono ad accedere al sistema. Al contrario, se la soglia è troppo bassa, potrebbero verificarsi **falsi negativi**, cioè persone autorizzate che vengono erroneamente escluse dal sistema. Pertanto, è essenziale trovare un **equilibrio**, e la soglia deve essere impostata in modo tale da ridurre al minimo entrambe le situazioni.

Qualità della Misurazione

La precisione dei sistemi biometrici è influenzata dalla **qualità della misurazione**. In passato, le impronte venivano acquisite tramite inchiostro, ma oggi vengono utilizzati dispositivi in grado di acquisire più volte l'impronta per ottenere una media, migliorando così la precisione. La **superficie acquisita** può variare a seconda di come viene posizionato il dito sul sensore. Per questo motivo, i sistemi moderni tendono a raccogliere più misurazioni per ridurre gli errori.

Variazioni Fisiche e Orientamento

Una delle **sfide** più comuni nel riconoscimento delle impronte digitali è legata alle **variazioni fisiche** che possono influire sulla qualità del dato acquisito, come **tagli**, **cicatrici** o **umidità** sulla pelle. Anche piccoli cambiamenti nell'orientamento del dito, come nel caso di una leggera **rotazione**, possono causare differenze nelle **coordinate** delle linee presenti sull'impronta, rendendo il confronto meno preciso. I sistemi moderni devono essere progettati per gestire queste variabilità, ad esempio utilizzando **coordinate polari** invece delle carte di **coordinate cartesiane**, che permettono una rappresentazione più flessibile dei dati biometrici.

Verifica e Riconoscimento Biometrico

Il **riconoscimento biometrico** può essere utilizzato per **verificare** l'identità di una persona o per **riconoscerla** in un sistema. La **verifica dell'identità** implica un confronto diretto tra un dato biometrico acquisito (ad esempio, l'impronta digitale) e uno precedentemente memorizzato per determinare se

appartengono alla stessa persona. Il **riconoscimento biometrico**, invece, cerca di **identificare** una persona tra una serie di soggetti memorizzati, senza bisogno di un dato biometrico pre-registrato.

Sistemi di Autenticazione a Più Fattori

Nel contesto della **sicurezza digitale**, spesso si utilizzano più livelli di **autenticazione**. Ad esempio, un sistema potrebbe richiedere una combinazione di **password**, **dati biometrici** e **token di sicurezza**, in modo da garantire una protezione robusta. Questo approccio viene utilizzato nei cosiddetti **sistemi di autenticazione a più fattori**, dove un utente deve provare la propria identità con più di una prova, come una password, un'impronta digitale e una chiave temporanea generata da un dispositivo o applicazione.

Sistemi con Ticket

Inoltre, esistono anche sistemi che utilizzano un processo a **ticket**, dove l'utente, dopo essere stato autenticato, riceve un "biglietto" che gli consente di accedere ai vari servizi in modo sicuro, senza dover ridigitare continuamente la password. Questi **ticket** hanno una durata limitata nel tempo, aumentando la sicurezza, poiché anche se un ticket venisse rubato, sarebbe inutile dopo la scadenza.

Conclusione

In sintesi, il **riconoscimento biometrico** è una tecnologia affascinante e potente, ma la sua implementazione deve considerare diversi fattori, come la **precisione** delle misurazioni, la **gestione della soglia di accettazione** e la protezione dei **dati sensibili**. L'equilibrio tra **facilità d'uso** e **sicurezza** è fondamentale per garantire un'esperienza utente ottimale e sicura.

4/12

Il protocollo Kerberos è un sistema di autenticazione utilizzato per garantire la sicurezza nelle comunicazioni tra client e server. Per comprendere il suo funzionamento, è utile descrivere i principali passaggi che caratterizzano il processo di autenticazione. Immaginiamo di avere un utente, A, che desidera comunicare con il TGS (Ticket Granting Server) attraverso un Authentication Server (AS). Questo processo prevede l'uso di vari messaggi e di chiavi condivise, garantendo che solo l'utente legittimo possa accedere ai servizi.

1. **Inizio della comunicazione:** L'utente A invia una richiesta al server di autenticazione (AS), desiderando ottenere un "ticket" per comunicare con il TGS. Questa richiesta include un marchio temporale (timestamp) per evitare attacchi di tipo replay, ossia che qualcuno intercetti e invii nuovamente il messaggio.
2. **Verifica dell'identità di A:** L'AS non si fida completamente dell'identità di A, quindi cifra il messaggio con la chiave condivisa tra A e l'AS. Questo assicura che solo A possa decrittare il messaggio e ottenere le informazioni necessarie per il passo successivo.
3. **Generazione del ticket:** L'AS invia a A un "authentication ticket" cifrato con una chiave condivisa tra AS e TGS. Questo ticket contiene un identificatore di sessione, un marchio temporale e altre informazioni, che permetteranno ad A di comunicare in modo sicuro con il TGS.
4. **Autenticazione con il TGS:** Una volta ricevuto il ticket, A si rivolge al TGS. A invia il ticket al TGS senza modificarlo, in quanto non è in grado di leggerlo (è cifrato). A invia inoltre una richiesta per parlare con il server B, cifrando il messaggio con la chiave di sessione condivisa tra A e il TGS (OutKey). Questo passaggio assicura che solo A possa inviare il messaggio, poiché solo A conosce la chiave.
5. **Verifica dell'autenticità da parte del TGS:** Il TGS decifra il ticket, estrae la chiave di sessione (OutKey) e la utilizza per verificare l'identità di A. Dopo aver autenticato A, il TGS invia un nuovo ticket cifrato con la chiave condivisa tra TGS e B, permettendo ad A di comunicare con B.
6. **Comunicazione con B:** A ora può inviare il ticket a B, che lo decifra utilizzando la chiave condivisa con il TGS. Questo permette a B di verificare che la richiesta provenga effettivamente da A, garantendo l'autenticità della comunicazione.

7. **Autenticazione di B:** A invia ulteriori informazioni a B per confermare la sua identità, utilizzando un altro marchio temporale e la chiave di sessione appropriata. B, a sua volta, verifica la validità di questi dati, confermando che la comunicazione è legittima.

Il processo descritto mostra come Kerberos utilizzi un sistema di chiavi condivise e marchi temporali per garantire la sicurezza e prevenire gli attacchi. L'uso di chiavi simmetriche (condivise) tra i vari server e client assicura che solo le parti autorizzate possano accedere ai dati sensibili, mantenendo il sistema protetto.

Il protocollo Kerberos è utilizzato in vari contesti, come nelle reti aziendali, e garantisce che le comunicazioni siano sicure, evitando che informazioni sensibili vengano intercettate o modificate da terzi non autorizzati. Sebbene il suo utilizzo non sia così comune come altri protocolli, rappresenta una base solida per la sicurezza nelle comunicazioni informatiche.

Il processo di comunicazione confidenziale in un sistema informatico richiede una chiave segreta condivisa tra le parti coinvolte. Per poter utilizzare questa chiave, è necessario un metodo iniziale di scambio sicuro della stessa. La difficoltà principale in questo processo risiede nel fatto che, inizialmente, le due parti non possiedono una chiave condivisa e devono trovarne una in modo sicuro.

Una soluzione a questo problema è rappresentata dal protocollo **Diffie-Hellman**, che permette a due parti di generare una chiave segreta condivisa senza doverla mai scambiare direttamente. In questo processo, ciascuna parte possiede una coppia di chiavi pubblica e privata. La chiave pubblica può essere condivisa apertamente, mentre la chiave privata è mantenuta segreta. Il protocollo funziona grazie alla difficoltà di calcolare il logaritmo discreto, un problema matematico che rende sicuro il processo di generazione della chiave condivisa.

Per applicare la cifratura, è necessario utilizzare un algoritmo di cifratura simmetrica, che utilizza la chiave condivisa per criptare i dati. Gli algoritmi di cifratura più comuni sono quelli che appartengono alla suite **IPsec**, che include il protocollo **ESP (Encapsulated Security Payload)**. Questo protocollo fornisce sia la cifratura dei dati (confidenzialità) che la firma dei dati (autenticazione). Esiste anche un altro protocollo in IPsec, chiamato **Authentication Header (AH)**, che fornisce solo l'autenticazione, garantendo l'integrità del messaggio ma non la sua confidenzialità.

Inoltre, per garantire la sicurezza della comunicazione, è necessario definire le **Security Associations (SA)**. Una **Security Association** è una relazione tra due entità che stabilisce le modalità di protezione delle comunicazioni. Queste informazioni sono memorizzate in un database chiamato **Security Policy Database (SPD)**, che contiene le regole di sicurezza per la gestione dei pacchetti. Quando un pacchetto arriva, il sistema consulta l'SPD per determinare se applicare la protezione tramite IPsec e quale algoritmo utilizzare.

Il protocollo **IKE (Internet Key Exchange)** viene utilizzato per negoziare e stabilire la chiave condivisa tra le due parti, che successivamente può essere utilizzata per cifrare i pacchetti di dati. Una volta stabilita la chiave, i pacchetti possono essere cifrati o autenticati a seconda delle necessità di confidenzialità e integrità.

Esistono due modalità principali in IPsec: **modalità tunnel** e **modalità trasporto**. Nella modalità tunnel, l'intero pacchetto IP (compreso l'intestazione) viene incapsulato in un nuovo pacchetto IP cifrato. Nella modalità trasporto, solo il payload del pacchetto (i dati veri e propri) viene cifrato.

Un altro aspetto importante è la gestione delle **tabelle di routing e delle policy di sicurezza**. Queste tabelle determinano se applicare la cifratura ai pacchetti in base alla destinazione e al tipo di protocollo. Ad esempio, i pacchetti destinati alla porta 443 (HTTPS) non necessitano di cifratura tramite IPsec, poiché sono già protetti da TLS.

Infine, la **Maximum Transfer Unit (MTU)** è un parametro fondamentale per il dimensionamento dei pacchetti. L'uso di IPsec, in particolare con la modalità tunnel, aumenta la dimensione del pacchetto a causa dell'overhead introdotto dalla cifratura e dalle intestazioni aggiuntive. Questo deve essere preso in considerazione per evitare la frammentazione dei pacchetti durante la trasmissione.

Per concludere, il processo di cifratura e scambio sicuro delle chiavi in IPsec dipende da una serie di protocolli e algoritmi matematici che lavorano insieme per garantire confidenzialità, integrità e autenticazione dei dati scambiati tra le entità in comunicazione.

10/12

La Sicurezza della Posta Elettronica

La **sicurezza della posta elettronica** è fondamentale in quanto rappresenta uno degli applicativi più diffusi nel contesto delle comunicazioni elettroniche. È stato uno dei primi sistemi a essere adottato in un periodo in cui la **sicurezza** non era una priorità, con il risultato che il sistema è nato senza misure adeguate di protezione. Questo è simile a quanto accaduto con il **PPP** (Point-to-Point Protocol), che è stato sviluppato senza un controllo sulla sicurezza, concentrandosi piuttosto sulla capacità di connettersi. Così, la **posta elettronica** è cresciuta e si è evoluta senza una protezione di base, rendendo essenziale l'introduzione di misure di sicurezza nel tempo.

Applicativi con Progettazione Sicura

Al contrario, altri applicativi, come i **database**, hanno visto fin da subito una progettazione orientata alla sicurezza. Così anche per le comunicazioni web, come **HTTPS**, che è stato sviluppato per garantire la sicurezza. Nel corso di questo studio, abbiamo esplorato protocolli come **IPsec** e **TLS**, strumenti generici che assicurano la sicurezza nelle comunicazioni, indipendentemente dall'applicativo utilizzato. Ma, per la **posta elettronica**, è fondamentale affrontare la sicurezza anche a livello **applicativo**, visto che è un servizio molto diffuso e critico.

Funzionamento della Posta Elettronica

La posta elettronica funziona attraverso vari elementi: c'è l'**utente** che invia la posta, quello che la riceve, e i **server** che si occupano di trasferirla. I **Mail Transfer Agents** (MTA), come i server **SMTP**, sono responsabili della spedizione delle email. Questo solleva una domanda interessante: perché sono necessari diversi **server SMTP** se, a livello applicativo, la comunicazione dovrebbe avvenire direttamente da server a server? La risposta risiede nel fatto che, oltre alla semplice trasmissione, le email possono passare attraverso vari passaggi di verifica, come il controllo **anti-spam** e **antivirus**, prima di arrivare a destinazione.

MIME e Sicurezza della Posta

Il disegno classico della posta elettronica prevede che il contenuto del messaggio possa includere vari tipi di media, grazie al protocollo **MIME**. Ad esempio, un'email può contenere **testo**, **immagini**, **audio** e **video**, tutti codificati in formato testuale per poter essere inviati tramite la rete. Questo permette una grande flessibilità, ma comporta anche dei **rischi di sicurezza**, come la possibilità di **intercettazioni** o attacchi che compromettano l'**integrità** del messaggio.

Protezione della Posta Elettronica: Confidenzialità e Integrità

Per proteggere la **posta elettronica**, è necessario garantire la **confidenzialità** (per evitare che il messaggio venga letto da persone non autorizzate) e l'**integrità** (per garantire che il messaggio non venga alterato durante il trasporto). Inoltre, è importante difendersi dall'**attacco di spoofing**, che consente a un attaccante di mascherare la propria identità.

Strumenti di Sicurezza: STARTTLS e S/MIME

Per mitigare questi rischi, si utilizzano vari strumenti e tecniche. Ad esempio, **STARTTLS** è un protocollo che consente di aggiungere la cifratura alle connessioni di posta elettronica senza cambiare la porta di connessione. In pratica, le connessioni vengono inizialmente stabilite in chiaro, ma poi vengono upgradeate a una connessione sicura tramite **TLS**, proteggendo la comunicazione.

Un altro strumento importante è **S/MIME** (Secure/Multipurpose Internet Mail Extensions), che estende MIME per consentire la cifratura e la **firma digitale** dei messaggi. Con **S/MIME**, non solo è possibile inviare contenuti multimediali, ma si può anche garantire l'**autenticità** e l'**integrità** del messaggio. Un messaggio

firmato digitalmente può essere verificato dal destinatario per assicurarsi che non sia stato alterato, e la firma garantisce che provenga dalla persona che dice di essere.

La domanda che ci poniamo riguarda il motivo per cui non si utilizza solo la **crittografia simmetrica** per cifrare i messaggi. La **crittografia simmetrica** è un tipo di cifratura che usa la stessa **chiave** sia per cifrare che per decifrare i dati. Mentre la **crittografia simmetrica** è molto veloce, il problema principale è come scambiare in modo sicuro la **chiave segreta** tra il mittente e il destinatario.

Per risolvere questo problema, viene usato un sistema di **crittografia asimmetrica**, che si basa su due **chiavi**: una **pubblica** e una **privata**. La **chiave pubblica** è condivisa con chiunque, mentre la **chiave privata** è tenuta segreta dal destinatario. In questo sistema, il mittente può cifrare un messaggio utilizzando la **chiave pubblica** del destinatario, ma solo il destinatario, con la sua **chiave privata**, può decifrarlo.

In un sistema di comunicazione sicura, quindi, la **crittografia asimmetrica** è utilizzata per cifrare la **chiave simmetrica**. In pratica, il mittente cifra il messaggio usando una **chiave simmetrica** (che è veloce), e poi cifra anche questa **chiave** con la **chiave pubblica** del destinatario. In questo modo, solo il destinatario, che possiede la sua **chiave privata**, potrà decifrare la **chiave simmetrica** e usare quella per decifrare il messaggio. Questo approccio è molto più efficiente rispetto all'uso esclusivo della **crittografia asimmetrica** per cifrare l'intero messaggio, che sarebbe troppo lento.

Oltre alla **confidenzialità**, è fondamentale garantire anche l'**integrità** e l'**autenticità** del messaggio. Per fare ciò, si utilizza una **firma digitale**. La **firma digitale** è creata utilizzando la **chiave privata** del mittente e permette al destinatario di verificare che il messaggio non sia stato alterato e che provenga realmente dal mittente dichiarato. In questo modo, se il messaggio è stato modificato, la **firma digitale** non corrisponderà più, e il destinatario lo rileverà.

In sintesi, il processo di cifratura in un sistema sicuro funziona così:

1. Il mittente cifra il messaggio con una **chiave simmetrica**.
2. La **chiave simmetrica** viene cifrata con la **chiave pubblica** del destinatario.
3. Il destinatario usa la sua **chiave privata** per decifrare la **chiave simmetrica**.
4. Infine, la **chiave simmetrica** viene usata per decifrare il messaggio.

Questo sistema combina la velocità della **crittografia simmetrica** con la sicurezza della **crittografia asimmetrica**, garantendo che solo il destinatario possa leggere il messaggio e che la comunicazione sia sicura e autentica.

11/12

Risoluzione dei nomi e configurazione della rete

Quando si configura la rete di una macchina, bisogna inserire **quattro informazioni principali**:

1. **Indirizzo IP**: ogni macchina in una rete ha un proprio indirizzo IP.
2. **Maschera di rete**: permette di determinare quali indirizzi IP appartengono alla stessa rete.
3. **Gateway**: il router che consente alla macchina di comunicare con altre reti.
4. **DNS** (Domain Name System): serve per risolvere i nomi di dominio in indirizzi IP.

Se si vuole configurare un **resolver DNS** sulla macchina, anche il software di risoluzione dei nomi deve essere configurato correttamente per sapere a quale **DNS** chiedere la risoluzione. Questo avviene quando si configurano i parametri di rete, come l'IP della macchina, la sua maschera di rete, il gateway e l'indirizzo del **DNS** a cui fare riferimento.

Quando la macchina fa una richiesta di risoluzione di un dominio come www.example.com, il resolver cerca di tradurre il nome in un indirizzo IP. La risoluzione dei nomi può avvenire in due modalità:

- **Ricorsiva**: il resolver DNS contatta altri server DNS finché non trova la risposta.

- **Iterativa:** il resolver chiede a un server DNS di fare il primo passo, ma poi sarà la macchina a fare il resto del lavoro.

Se la macchina ha già risolto un nome di dominio in precedenza, la risposta verrà memorizzata in **cache** per velocizzare le future richieste.

Sicurezza nella risoluzione dei nomi

Durante la risoluzione dei nomi, esistono dei rischi di attacco, come l'**attacco man-in-the-middle** o l'**impersonificazione**. In questi casi, un attaccante potrebbe intercettare o falsificare le risposte DNS, indirizzando la macchina verso un sito falso.

Per proteggersi, si può utilizzare **DNSSEC** (DNS Security Extensions), un'estensione che protegge le risposte DNS con **firma digitale**. In questo modo, ogni risposta DNS è autenticata e garantita in termini di **integrità**.

DNSSEC e DANE

Il protocollo **DNSSEC** si basa sulla firma digitale per garantire che le risposte DNS siano autentiche. Tuttavia, c'è un ulteriore livello di protezione chiamato **DANE** (DNS-Based Authentication of Named Entities), che si integra con DNSSEC per l'autenticazione dei certificati **TLS**.

In pratica, con DANE non ci si affida a **CA** (Certificate Authorities) per la verifica dei certificati TLS, ma si usa il DNS per ottenere le informazioni necessarie. Questo aggiunge una sicurezza maggiore, poiché permette di verificare l'autenticità dei certificati direttamente tramite il DNS.

Autenticazione e protezione delle email

- **SPF** (Sender Policy Framework) è un protocollo che permette di verificare se un messaggio di posta proviene da un server autorizzato a inviare email per un dominio specifico. In caso contrario, il messaggio può essere respinto o messo in quarantena.
- **DKIM** (DomainKeys Identified Mail) aggiunge una **firma crittografica** ai messaggi di posta elettronica per garantirne l'autenticità. Il server ricevente verifica la firma per assicurarsi che il messaggio non sia stato manomesso.
- **DMARC** (Domain-based Message Authentication, Reporting & Conformance) si integra con SPF e DKIM per definire politiche su come trattare i messaggi sospetti, ad esempio mettendoli in quarantena o rifiutandoli.

Processo di verifica dei messaggi e-mail

Quando un server riceve una email, verifica prima il record **SPF** del dominio di origine. Se il record SPF non è valido, il server può applicare politiche definite in **DMARC**, come mettere il messaggio in quarantena o rifiutarlo.

Inoltre, se il messaggio è firmato con **DKIM**, il server verifica la firma per confermare l'autenticità del mittente. Se entrambe le verifiche falliscono, il messaggio può essere scartato o messo in quarantena.

Gestione dei messaggi sospetti

I messaggi che non superano i controlli SPF e DKIM vengono gestiti seguendo le politiche DMARC. Queste politiche possono includere azioni come:

- **Quarantena:** i messaggi sospetti vengono spostati in una cartella separata per ulteriori verifiche da parte di un amministratore.
- **Rifiuto:** i messaggi vengono scartati senza ulteriori verifiche.

L'uso di **DMARC** permette ai domini di definire come devono essere trattati i messaggi non autenticati, prevenendo attacchi di **spoofing** e garantendo la sicurezza delle comunicazioni via e-mail.

Conclusione

In sintesi, l'implementazione di misure di sicurezza come **DNSSEC**, **DANE**, **SPF**, **DKIM**, e **DMARC** è fondamentale per proteggere le comunicazioni online. Questi protocolli garantiscono l'integrità, l'autenticità e la sicurezza dei dati, prevenendo attacchi come l'**impersonificazione** e il **phishing**

Programma Esame:

1. **ipSEC e TLS**: Differenze tra BISEC e TLS.
2. Spiegazione del protocollo **ipSEC**.
3. Confronto tra ipSEC e **TLS**.
4. **SPD**: Cos'è la **SPD** e come funziona.
5. Autenticazione: tipi di autenticazione.
6. Come funziona Kerberos, che tipo di protocollo è
7. **Ruolo di Creatore**: Compiti e responsabilità di chi svolge il ruolo di creatore.
8. **Wireshark**: Uso di Wireshark nel contesto della sicurezza di rete
9. **Funzione DLS**: Spiegazione della funzione **DLS**.
10. **HTTPS**: Differenze tra **HTTPS** e altre tecnologie di sicurezza (come il Port4One).
11. **DKIM e SPF**: Differenze tra i protocolli **DKIM** e **SPF**.
12. **Sicurezza di Rete**: Concetti legati alla sicurezza delle reti, inclusi i protocolli e gli strumenti di cifratura utilizzati nel **Wi-Fi**.
13. **IDS**: Cos'è un **IDS** e i tipi di **IDS** conosciuti.
14. Cos'è un **firewall** e a che livelli lavora